

2. НАСТРОЙКА РАБОЧЕГО ОКРУЖЕНИЯ

В ходе прохождения учебной практики использовался язык программирования Python, среда разработки (IDE) PyCharm.



Рисунок 2.1 – Логотип PyCharm

2.1 Описание языка. Достоинства и недостатки. Описание используемой IDE.

2.1.1 Описание языка программирования Python. Достоинства и недостатки.

Python – это высокоуровневый язык программирования, с помощью которого создают сайты, разрабатывают приложения, автоматизируют процессы анализа или визуализации данных. Python не был разработан для конкретных целей, поэтому подходит как для создания алгоритма рекомендаций видеосервиса, так и для разработки программного обеспечения для самоуправляемых автомобилей или управления космическими аппаратами на других планетах.

К основным характеристикам Python относят:

- а) объектную ориентированность;
- б) читабельность кода;
- в) интерпретируемость;
- г) динамическую типизацию.

Преимущества Python:

- Лёгкость освоения.
- Простота визуального воспитания.
- Кроссплатформенность.
- Скорость разработки.
- Универсальность.
- Множество инструментов.
- Масштабируемость.

Недостатки Python:

- Медленная работа.
- Трудность переноса проектов на другие системы.
- Ресурсоёмкость [1].

Таким образом, выбор языка программирования Python является обоснованным благодаря его универсальности, простоте освоения и широким возможностям применения. Несмотря на отдельные недостатки, такие как сравнительно низкая скорость выполнения программ и высокая ресурсоёмкость, преимущества Python существенно перевешивают его ограничения. Высокая читабельность кода, динамическая типизация и

большое количество готовых библиотек позволяют значительно ускорить процесс разработки и упростить сопровождение программных продуктов. Кроме того, кроссплатформенность и масштабируемость делают Python удобным инструментом для реализации проектов различной сложности. Именно поэтому данный язык программирования выбран в качестве основного средства разработки в рамках данной работы.

2.1.2 Описание используемой IDE.

PyCharm – это программное обеспечение, которое представляет собой интегрированную среду разработки для языка программирования Python.

Использование PyCharm предоставляет ряд преимуществ по сравнению с другими средствами разработки. Рассмотрим основные плюсы использования данной IDE для программирования.

- мощные инструменты подсказок и автодополнения, которые позволяют увеличить производительность и качество кода;
- интегрированная система рефакторинга позволяет легко изменять структуру кода, делая его более читаемым и понятным;
- отличная интеграция с инструментами управления версиями, такими как Git, что упрощает работу в команде;
- интуитивно понятный интерфейс, который упрощает навигацию по проекту;
- широкий выбор плагинов для настройки среды под свои потребности;
- интеллектуальный анализ кода поможет выявить потенциальные проблемы и предложить варианты их решения;

- возможность использования отладчика для пошагового анализа выполнения кода;
- поддержка различных фреймворков, библиотек и инструментов для анализа кода.

Все эти особенности делают PyCharm незаменимым инструментом для разработчиков на Python, помогая им создавать высококачественное программное обеспечение быстро и эффективно [2].

2.2 Описание используемых библиотек.

В процессе разработки программного средства были выбраны стандартные и сторонние библиотеки языка Python. Основной библиотекой для создания графического интерфейса пользователя является tkinter. Она применяется для построения окон, кнопок, надписей, областей отображения, а также для обработки действий пользователя. С помощью данной библиотеки реализованы главное меню, экран выбора параметров игры, поле трассы и элементы управления игровым процессом.

Для работы с файлами конфигурации и описаниями трасс выбрана библиотека json. Она позволяет сохранять и загружать данные в удобном структурированном формате. С её помощью в проекте хранятся координаты стен, стартовых и финишных позиций, контрольных точек и других параметров трассы.

Библиотека os применяется для работы с файловой системой. Она используется при формировании путей к игровым трассам, файлам превью и конфигурационным данным, а также для проверки существования необходимых файлов.

Для загрузки и изменения размеров изображений превью трасс выбрана библиотека Pillow (PIL). Она расширяет возможности стандартного Python по работе с графическими файлами и позволяет открывать изображения формата PNG, изменять их размеры и подготавливать для отображения в интерфейсе программы.

Библиотека copy применяется для создания копий игровых данных. Это необходимо для корректной работы с состоянием игры, сохранениями и повторной инициализацией объектов без изменения исходных структур данных.

В вычислительной части программы нужна библиотека math. Она необходима для выполнения математических операций, связанных с обработкой движения объектов, определением траектории и расчётом столкновений.

Также в проекте пригодятся модули dataclasses и typing. Модуль dataclasses упрощает описание структур данных, используемых для хранения результатов хода и столкновений. Модуль typing применяется для повышения читаемости кода и более точного описания типов данных.

Таким образом, выбранные библиотеки обеспечивают реализацию графического интерфейса, обработку игровых данных, работу с файлами, загрузку изображений и выполнение вычислений, необходимых для функционирования приложения.

Скриншоты работы в выбранной среде приведены на рисунке 2.2.

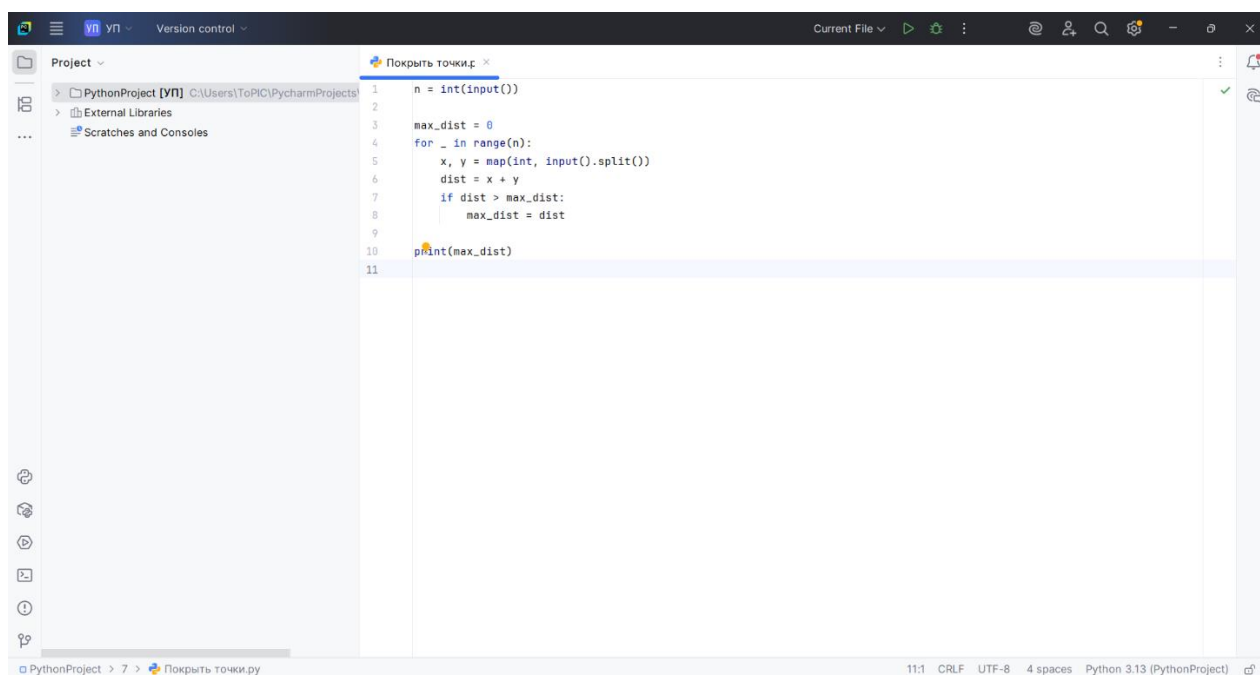


Рисунок 2.2 – Интерфейс PyCharm

Для системы контроля версий в соответствии с заданием использована система GitHub.

Ссылка на профиль: <https://github.com/ToPIC-pro>.

Ссылка на репозиторий: <https://github.com/ToPIC-pro/RaceTrack-UP.05>.

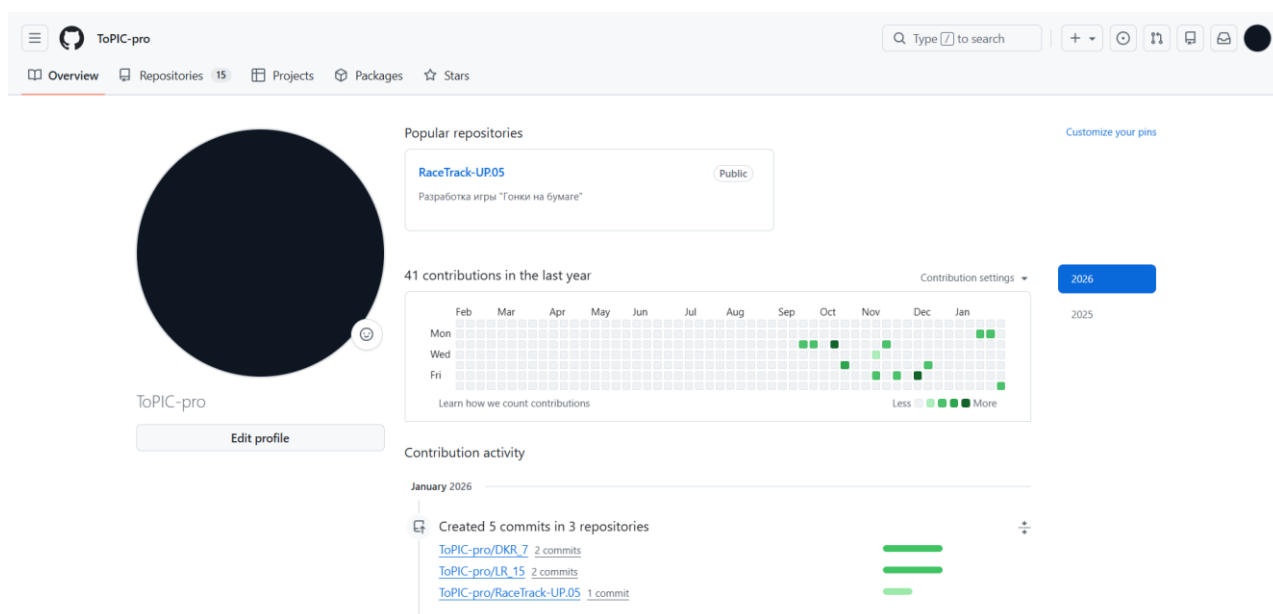


Рисунок 2.3 – Профиль GitHub

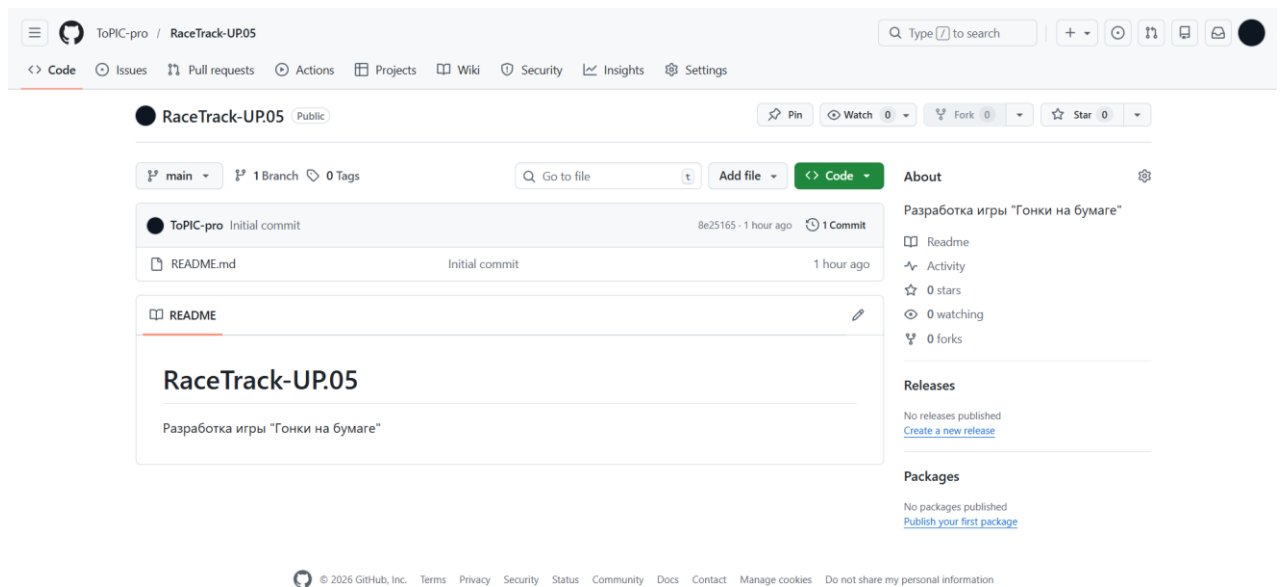


Рисунок 2.4 – Репозиторий для версий игры

4 ОПИСАНИЕ ВЫПОЛНЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

4.1 Анализ предметной области

Игра «Гонки на бумаге» – это пошаговая логическая игра, реализуемая на клетчатой бумаге, в которой моделируется движение гоночного автомобиля по трассе с учётом скорости, направления и инерции. Данная игра относится к классу бумажных игр с дискретным пространством состояний и используется как в развлекательных, так и в образовательных целях, в частности для иллюстрации понятий векторного движения, кинематики и алгоритмического мышления [3, 7].

Игровое поле представляет собой трассу, произвольно нарисованную на клетчатой бумаге, ограниченную границами. Каждый игрок управляет «машиной», положение которой задаётся точкой на сетке. Основная особенность игры заключается в том, что ход игрока определяется не абсолютным перемещением, а изменением вектора скорости, что приближает модель игры к реальному движению объектов с инерцией [4]. Возможное перемещение точки приведено на рисунке 4.1.

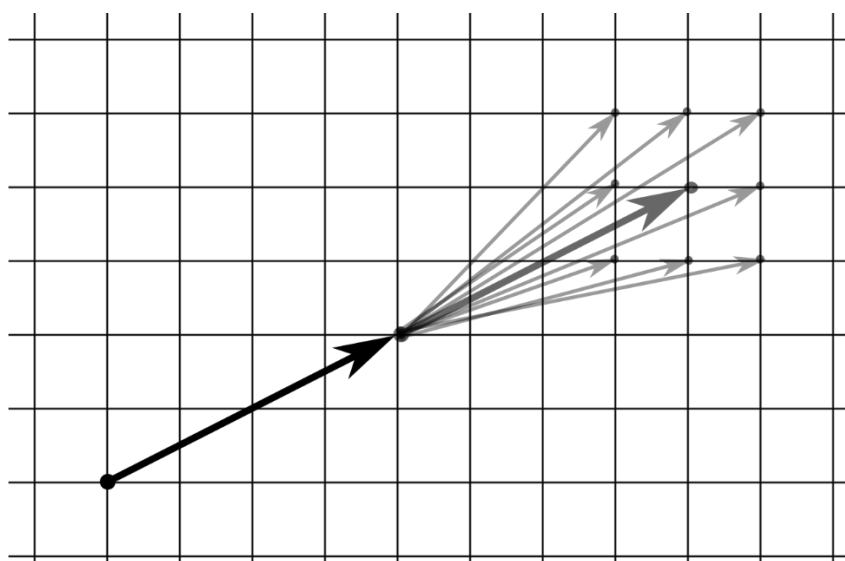


Рисунок 4.1 – Возможное перемещение точки

С формальной точки зрения состояние игры в каждый момент времени может быть описано тремя параметрами:

- текущие координаты автомобиля;
- текущий вектор скорости;
- конфигурация трассы и её ограничений.

При каждом ходе игрок выбирает новое ускорение, изменяя вектор скорости, после чего автомобиль перемещается на соответствующее расстояние. Если траектория пересекает границу трассы, считается, что автомобиль сошёл с дистанции или потерпел аварию, в зависимости от выбранных правил [3].

Несмотря на внешнюю простоту, игра «Гонки на бумаге» обладает высокой комбинаторной сложностью. Количество возможных состояний растёт экспоненциально с увеличением длины трассы и допустимого диапазона скоростей. В научных исследованиях данная игра известна под названием *Racetrack* и рассматривается как задача алгоритмического планирования и оптимизации траектории. В работе [4] показано, что поиск оптимального пути на гоночной трассе с ограничениями относится к классу вычислительно сложных задач и требует применения эвристических или приближённых алгоритмов.

В практическом и популярном контексте игра часто используется как средство развития логического мышления и интуитивного понимания физических процессов. Описания правил, примеры игровых партий и различные вариации игры широко представлены в источниках, ориентированных на массовую аудиторию [3, 5, 6, 7]. В таких материалах подчёркивается обучающий потенциал игры, связанный с прогнозированием движения, анализом последствий решений и планированием на несколько ходов вперёд.

Для обобщённой оценки распространённости и жизненного цикла цифровых игр «Гонки на бумаге» были проанализированы открытые данные магазинов мобильных приложений, игровых порталов и пользовательских

сообществ. Поскольку точные статистические данные не всегда публикуются разработчиками, в работе используются усреднённые и оценочные показатели.

Большинство игр находятся в свободном доступе от 5 до 10 лет. Алгоритмические и учебные реализации (например, Racetrack) существуют значительно дольше – более 20 лет – в виде учебных проектов и исследовательских симуляторов. Средний срок «жизни» цифрового аналога можно оценить в 8-12 лет.

Мобильные аналоги, размещённые в Google Play и App Store, как правило, имеют от 10 000 до 100 000 загрузок. Более популярные проекты достигают нескольких сотен тысяч установок, однако большинство игр данного жанра ориентированы на нишевую аудиторию и не выходят на массовый рынок.

Обновления для подобных игр выходят нерегулярно. В среднем активная поддержка проекта продолжается 2-4 года, после чего игра остаётся доступной, но развивается минимально. Исключение составляют образовательные и исследовательские реализации, которые обновляются в рамках учебных курсов или научных задач.

Они ориентированы преимущественно на:

- школьников и студентов;
- пользователей, интересующихся логическими и математическими играми;
- разработчиков и исследователей в области алгоритмов и ИИ.

Массовая игровая аудитория проявляет ограниченный интерес к подобным проектам, что объясняет умеренные показатели распространения.

Таким образом, предметная область данной работы охватывает дискретное моделирование движения, элементы теории игр и алгоритмического поиска, а также прикладные аспекты использования игры «Гонки на бумаге» как модели для анализа стратегий и прогнозирования

поведения системы в условиях ограниченного пространства и правил перехода между состояниями.

4.1.1 Анализ проблемы

Несмотря на простоту правил и доступность игры «Гонки на бумаге», при её формализации и анализе возникает ряд существенных проблем. Основной сложностью является необходимость точного моделирования движения объекта с инерцией в дискретном пространстве, ограниченном заранее заданной трассой. В отличие от классических настольных игр, где переходы между состояниями строго фиксированы, в «Гонках» каждый ход зависит не только от текущего положения игрока, но и от накопленного вектора скорости [1]. Один из примеров вариативности ходов изображен на рисунке 4.2.

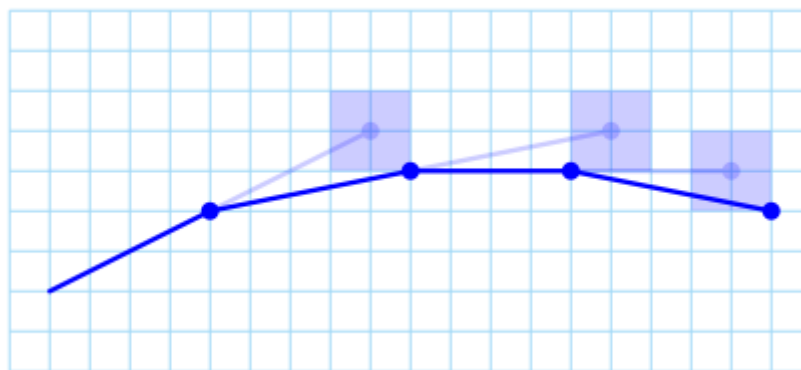


Рисунок 4.2 – Пример вариативности ходов

При попытке формального описания игры возникает проблема взрывного роста пространства состояний. Даже при ограниченном числе допустимых ускорений количество возможных траекторий движения автомобиля увеличивается нелинейно с ростом длины трассы. Это существенно усложняет задачи анализа, прогнозирования и поиска

оптимальной стратегии, особенно при автоматизации игрового процесса или создании интеллектуального агента [2].

Дополнительной проблемой является отсутствие единого набора правил. В различных источниках встречаются отличающиеся трактовки допустимых ускорений, обработки столкновений со стенами трассы, а также условий победы и поражения [1, 3, 4, 5]. Такое разнообразие правил затрудняет создание универсальной математической модели игры и требует предварительного выбора или стандартизации правил для дальнейшего анализа.

С практической точки зрения игра часто используется интуитивно, без строгого математического обоснования принимаемых решений. Большинство игроков планируют ходы, опираясь на визуальную оценку траектории, что не гарантирует оптимальности выбранного пути. В результате возникает задача формирования алгоритма рационального выбора хода, который учитывал бы текущую скорость, положение на трассе и возможные последствия в будущем.

В научных работах игра Racetrack рассматривается как задача поиска пути с ограничениями и частичной информацией. Показано, что даже в одиночном варианте игра может требовать сложных алгоритмов планирования, особенно если трасса открывается по мере движения или содержит узкие участки и резкие повороты [2]. Это делает задачу интересной с точки зрения теории алгоритмов, но одновременно усложняет её практическую реализацию.

Таким образом, основная проблема заключается в формализации и алгоритмизации игрового процесса таким образом, чтобы обеспечить корректное моделирование движения, учёт ограничений трассы и возможность прогнозирования дальнейшего поведения игрока. Решение данной проблемы требует разработки математической модели игры и выбора подходящих методов анализа траекторий и стратегий движения.

4.1.2 Обзор аналогов

В качестве аналогов игры «Гонки на бумаге» были рассмотрены цифровые реализации и игры, использующие схожие принципы дискретного движения, трассы с ограничениями и планирование траектории. Аналоги представлены мобильными, браузерными и алгоритмическими игровыми решениями.

1. PaperRacers (iOS)

PaperRacers – мобильная пошаговая гоночная игра для платформы iOS, основанная на идее движения объекта по трассе с учётом скорости и направления. Игроки поочерёдно выполняют ходы, изменяя траекторию движения автомобиля. Игра поддерживает многопользовательский режим и различные конфигурации трасс.



Рисунок 4.3 – PaperRacers

Преимущества:

- автоматизированная проверка допустимости ходов;
- многопользовательский режим;
- наглядная визуализация траектории движения.

Недостатки:

- ограниченная платформа распространения;
- упрощённая модель движения по сравнению с классической;
- ориентация на развлекательный формат.

Игра доступна в App Store на протяжении нескольких лет, имеет пользовательские рейтинги и отзывы. Точные данные о количестве загрузок не раскрываются, однако стабильное присутствие в магазине указывает на устойчивый интерес аудитории.

2. Paper Racing (Android)

Paper Racing Cars – мобильная игра для Android, использующая визуальный стиль рисования на бумаге. Игрок управляет автомобилем на трассах, создаваемых разработчиками или самими пользователями. Игра сочетает элементы пошагового и аркадного геймплея.



Рисунок 4.4 – Paper Racing

Преимущества:

- доступность на массовой мобильной платформе;
- возможность создания собственных трасс;
- простой и понятный интерфейс.

Недостатки:

- ограниченная глубина стратегии;
- часть функциональности доступна только в полной версии;
- слабый акцент на анализ траектории.

Приложение существует на рынке с середины 2010-х годов и имеет десятки тысяч установок, что свидетельствует о средней популярности среди мобильных пользователей.

3. Vector Racer (Android)

Vector Racer – мобильная игра, в которой реализована механика управления объектом через изменение вектора скорости. Перемещение осуществляется дискретно, что делает игру близкой к концепции «Гонок на бумаге» с точки зрения формальной модели движения.

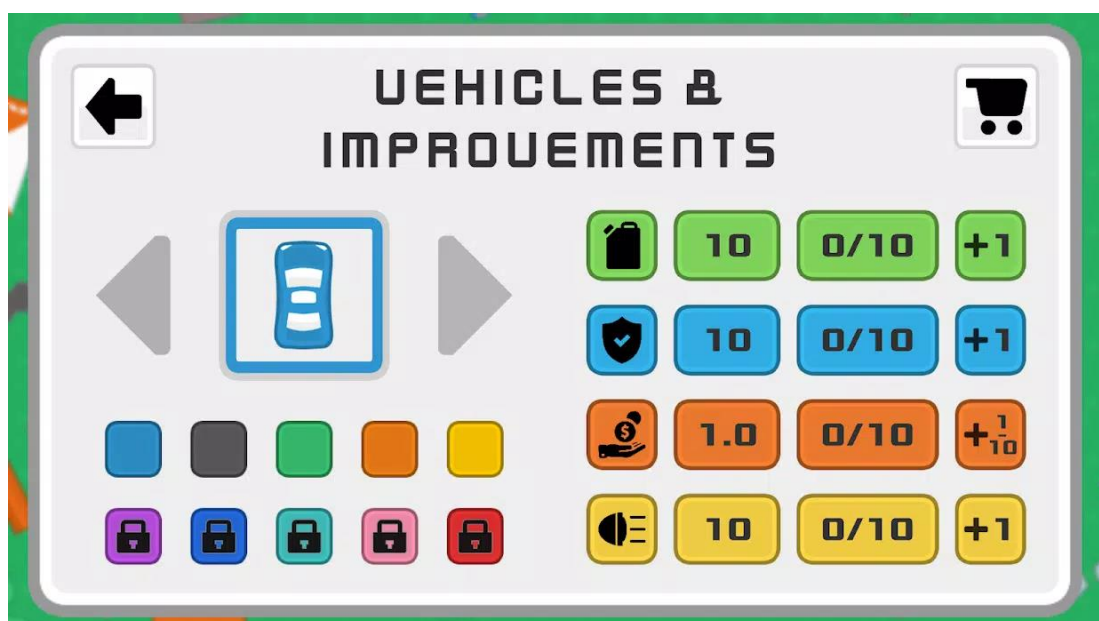


Рисунок 4.5 – Vector Racer (Android)

Преимущества:

- использование векторной модели скорости;
- пошаговый характер принятия решений;
- возможность кастомизации машин.

Недостатки:

- ограниченные возможности визуализации;
- отсутствие развитого пользовательского сообщества;
- редкие обновления.

Игра представлена в магазинах мобильных приложений несколько лет и ориентирована на узкую аудиторию пользователей, интересующихся логическими и математическими играми.

4. Racetrack (алгоритмическая игра-симуляция)

Racetrack – цифровая реализация одноимённой задачи, используемой в исследованиях по искусственному интеллекту и алгоритмическому планированию. Игра моделирует движение объекта по трассе с инерцией и ограничениями, где требуется найти оптимальную траекторию.

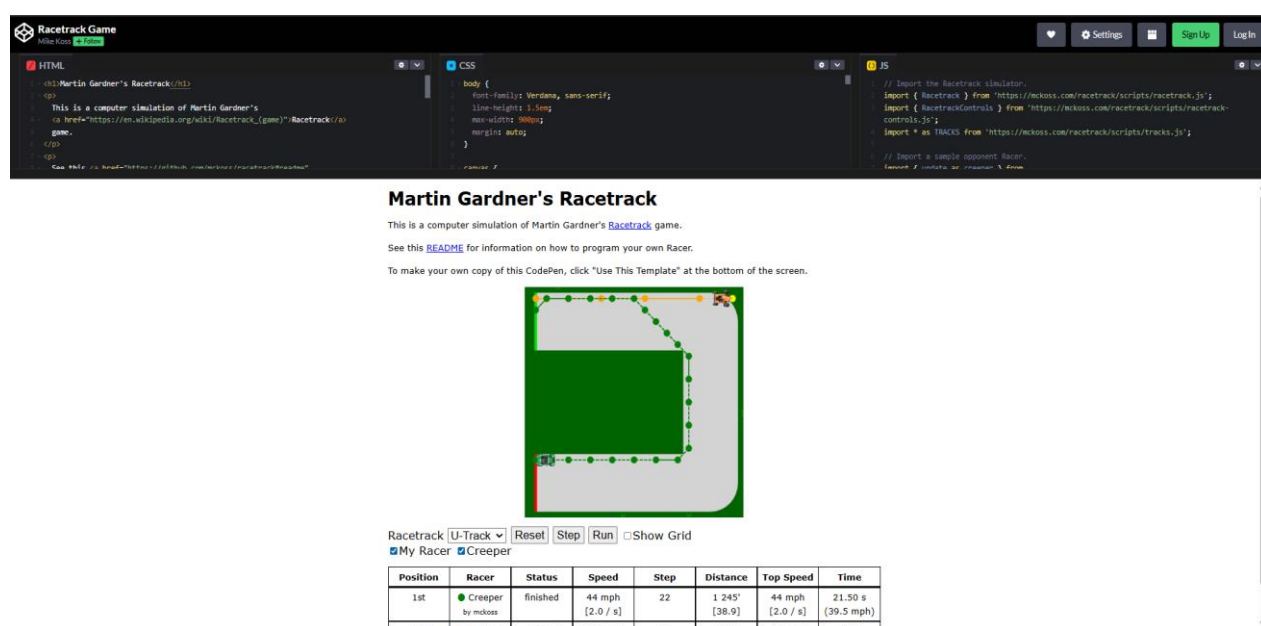


Рисунок 4.6 – Racetrack

Преимущества:

- строгая математическая модель;
- возможность формального анализа и оптимизации;
- применимость в образовательных и научных целях.

Недостатки:

- сложность восприятия для неподготовленного пользователя;
- отсутствие развлекательных элементов;
- ориентация на исследовательские задачи.

Подобные реализации используются в научных и учебных проектах на протяжении нескольких десятилетий и широко представлены в университетских курсах и публикациях.

5. Vector Racer (браузерная версия)

Vector Racer – браузерная пошаговая гоночная игра, реализующая классическую механику «Гонок на бумаге». Управление движением автомобиля осуществляется через изменение вектора скорости на клетчатом поле. Игрок последовательно выбирает направление и величину перемещения, стремясь пройти трассу и достичь финиша за минимальное число ходов. Игра не требует установки и запускается непосредственно в веб-браузере.

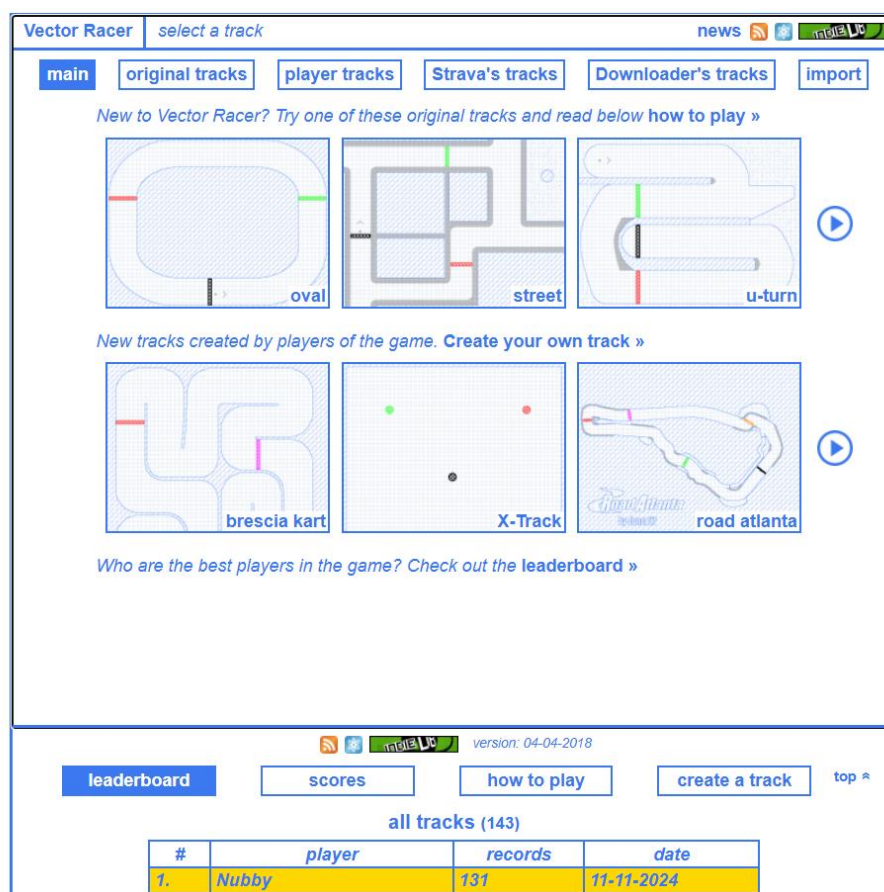


Рисунок 4.3 – Меню игры Vector Racers

Преимущества:

- реализация векторной модели движения, близкой к классическим «Гонкам на бумаге»;
- пошаговый характер игрового процесса, позволяющий планировать траекторию;
- отсутствие необходимости установки и кроссплатформенность;
- простота интерфейса и наглядность представления состояния игры.

Недостатки:

- минималистичная визуализация и отсутствие развитого графического оформления;
- ограниченный набор трасс и игровых режимов;
- ориентация преимущественно на узкую аудиторию пользователей, интересующихся логическими и алгоритмическими играми.

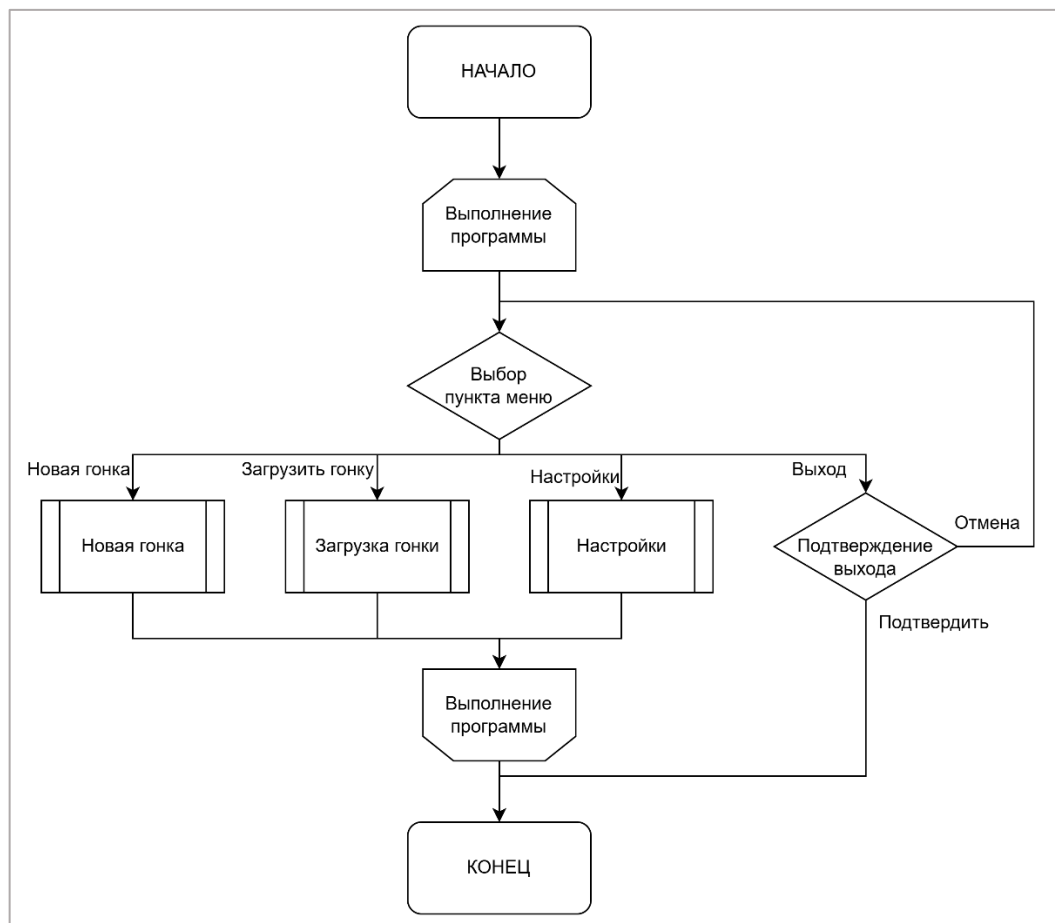
Vector Racer можно рассматривать как цифровой аналог классической бумажной игры, ориентированный не на развлекательную составляющую, а на точное воспроизведение механики движения с инерцией и планирования траектории.

Рассмотренные цифровые аналоги демонстрируют различные подходы к реализации идей, заложенных в игре «Гонки на бумаге». Большинство решений ориентированы на упрощение и визуальную привлекательность, что снижает возможности формального анализа и стратегического планирования. Это подтверждает актуальность разработки программной реализации, сохраняющей строгую логику движения и предоставляющей инструменты для анализа траекторий и принятия решений.

4.2 Разработка структуры приложения и алгоритмов функционирования

4.2.1 Разработка алгоритмов функционирования

После запуска приложения происходит инициализация интерфейса: загружается главное окно со всеми элементами управления и кнопками навигации. Пользователю отображается главный экран с возможностью начать новую игру, загрузить предыдущие (сохраненные) или выйти из приложения. Алгоритм работы главного окна представлен на рисунке 4.2.1.



4.2.1 – Алгоритм работы главного окна игры «Гонки на бумаге»

При переходе в режим выбора настроек пользователю предоставляется возможность изменить параметры приложения, включая смену оформления и языка интерфейса. После внесения необходимых изменений пользователь может перейти в главное меню. Алгоритм режима представлен на рисунке 4.2.2.

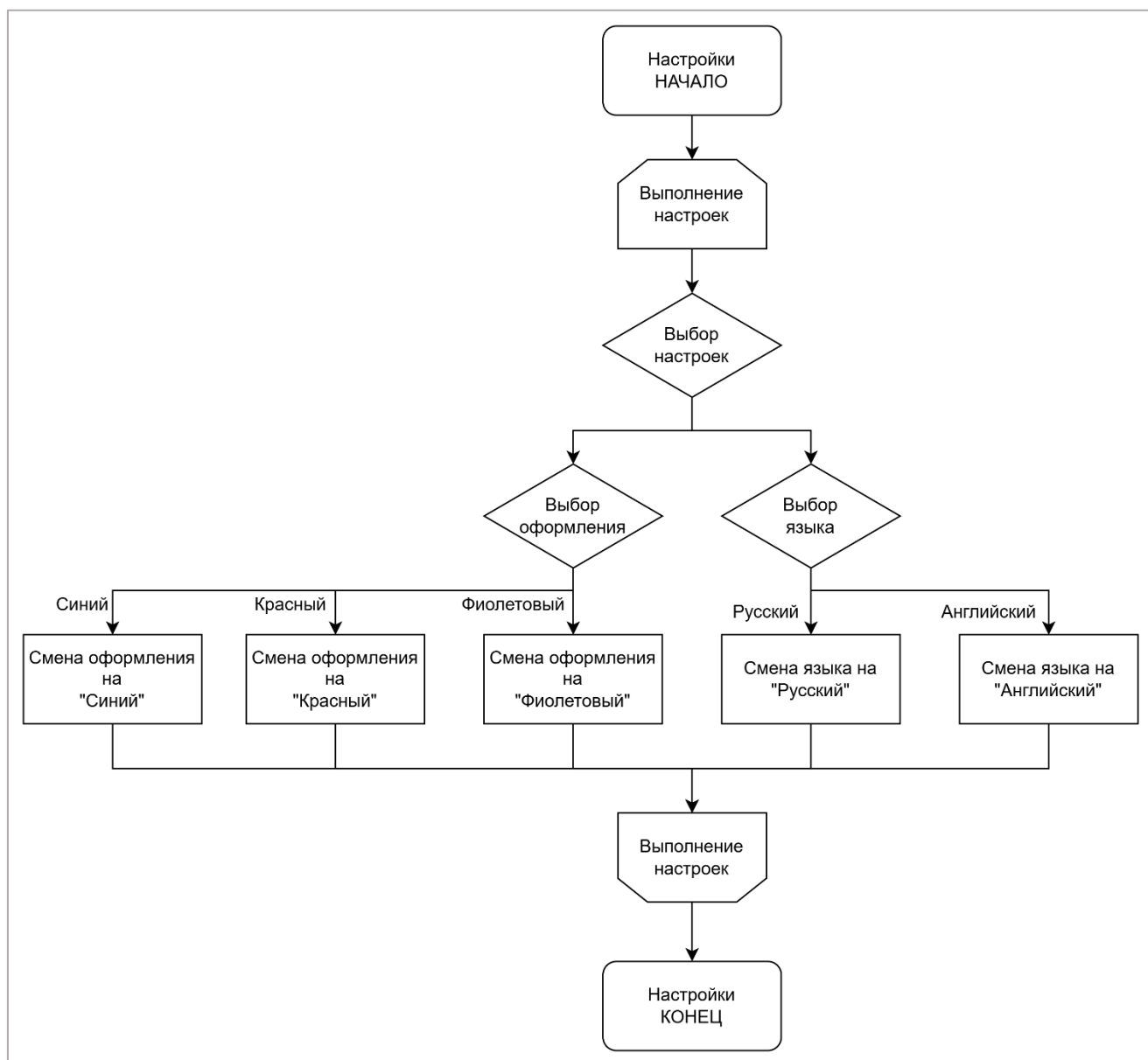


Рисунок 4.2.2 – Алгоритм режима «Настройки»

При выборе пункта «Загрузить гонку» пользователь переходит к экрану с предыдущими играми. Он может удалить их, или продолжить игру. Алгоритм пункта «Загрузить гонку» представлен на рисунке 4.2.3.

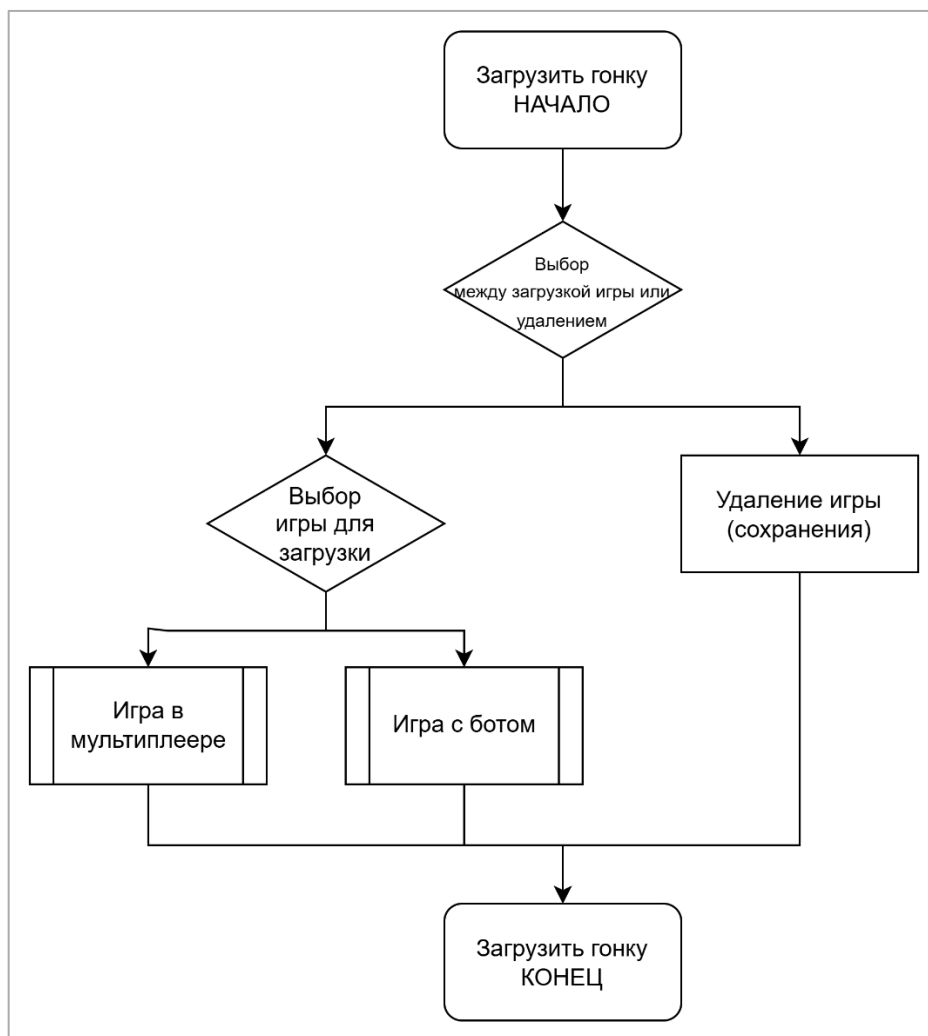


Рисунок 4.2.3. – Алгоритм пункта «Загрузить гонку»

При выборе пункта «Новая игра» пользователь переходит к экрану настройки параметров игры, где может выбрать режим (игра с ботом или игра с другими игроками). Алгоритм пункта «Новая гонка» представлен на рисунке 4.2.4.



Рисунок 4.2.4 – Алгоритм пункта «Новая гонка»

При выборе режима «Бот» пользователь задаёт уровень сложности, выбирает трассу и цвета для бота и для себя. Алгоритм настройки представлен на рисунке 4.2.5.

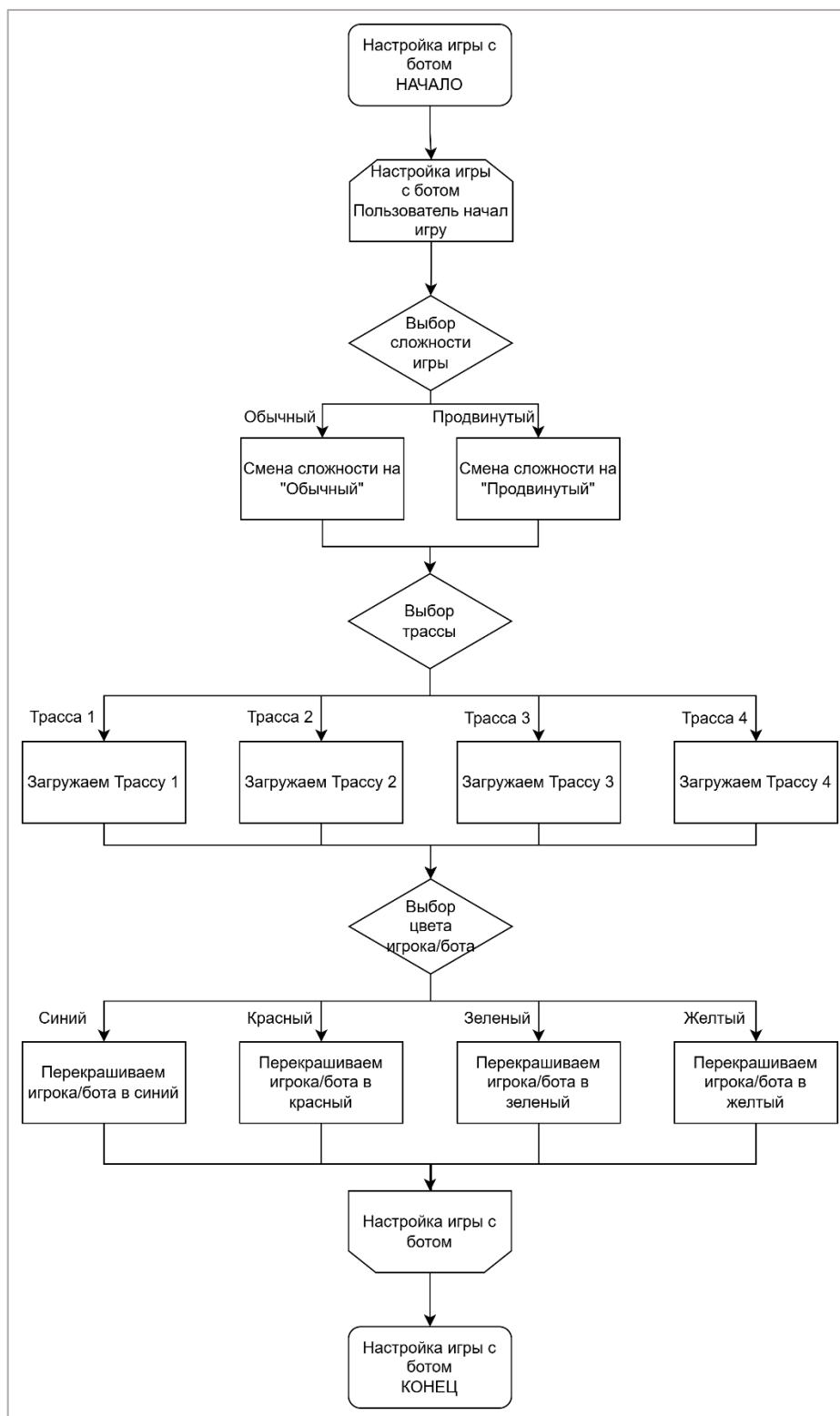


Рисунок 4.2.5 – Настройка игры с ботом

После настройки игры с ботом начинается игровой процесс. Игрок и бот поочерёдно выполняют ходы с проверкой условия завершения игры после каждого хода. При достижении определенных условий игра завершается. Алгоритм данного режима представлен на рисунке 4.2.6.

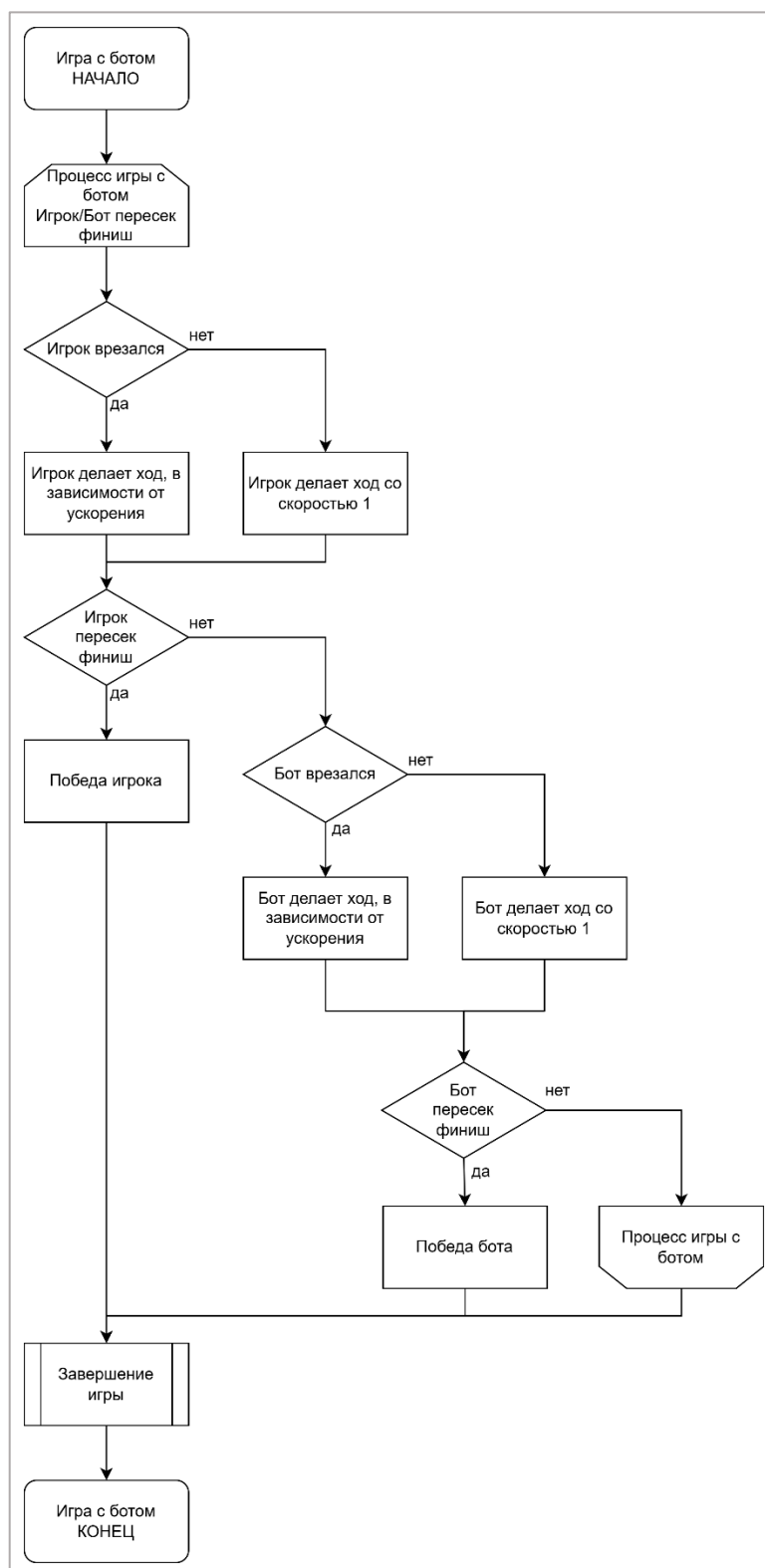


Рисунок 4.2.6 – Алгоритм режима «Бот»

При выборе режима «Локальный Мультиплеер» пользователи начинают настройку игры в мультиплеере. Они выбирают трассу, количество игроков, а также свои цвета. Алгоритм настройки игры в мультиплеере представлен на рисунке 4.2.7.



Рисунок 4.2.7 – Настройка игры в мультиплеере

После настройки игры в мультиплеере игроки поочередно выполняют ходы в соответствии с правилами игры «Гонки на бумаге», при этом после каждого хода осуществляется проверка условия завершения игры. Алгоритм режима «Локальный мультиплеер» представлен на рисунке 4.2.8.

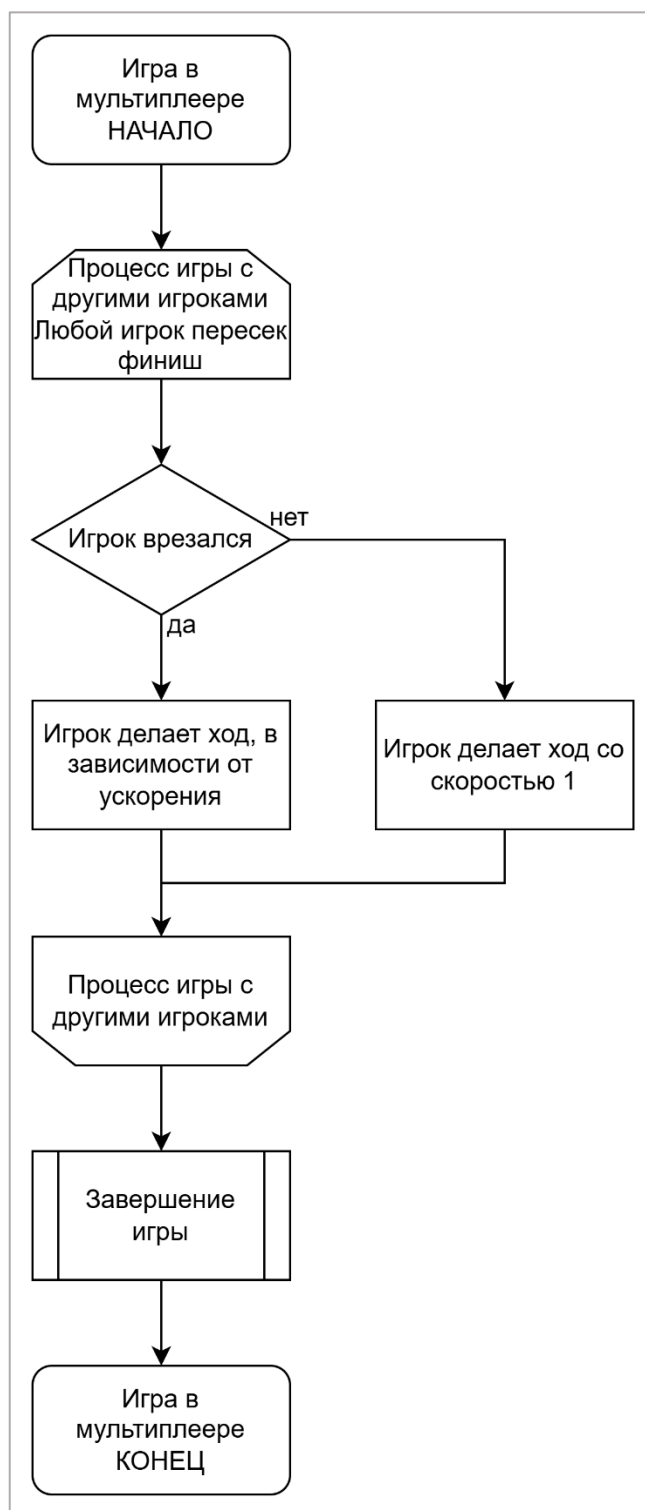


Рисунок 4.2.8 – Алгоритм режима «Локальный Мультиплеер»

После окончания игры выполняется вывод результатов игры. Пользователю предлагается выбрать дальнейшее действие: начать новую игру либо вернуться в главное меню. В зависимости от выбранного варианта осуществляется переход к соответствующему экрану, после чего режим завершения игры, представленный на рисунке 4.2.9, считается выполненным.

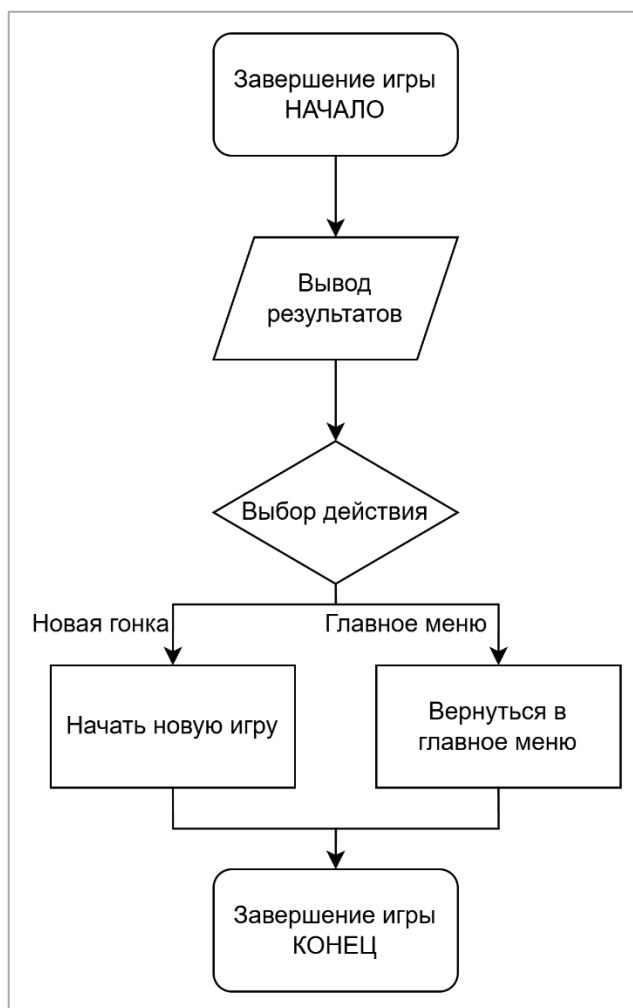


Рисунок 4.2.9 – Алгоритм режима «Завершение игры»

4.2.3 Проектирование интерфейса

Приводится прототип интерфейса, нарисованный в Figma. Прототип представлен на рисунке 4.3.1.

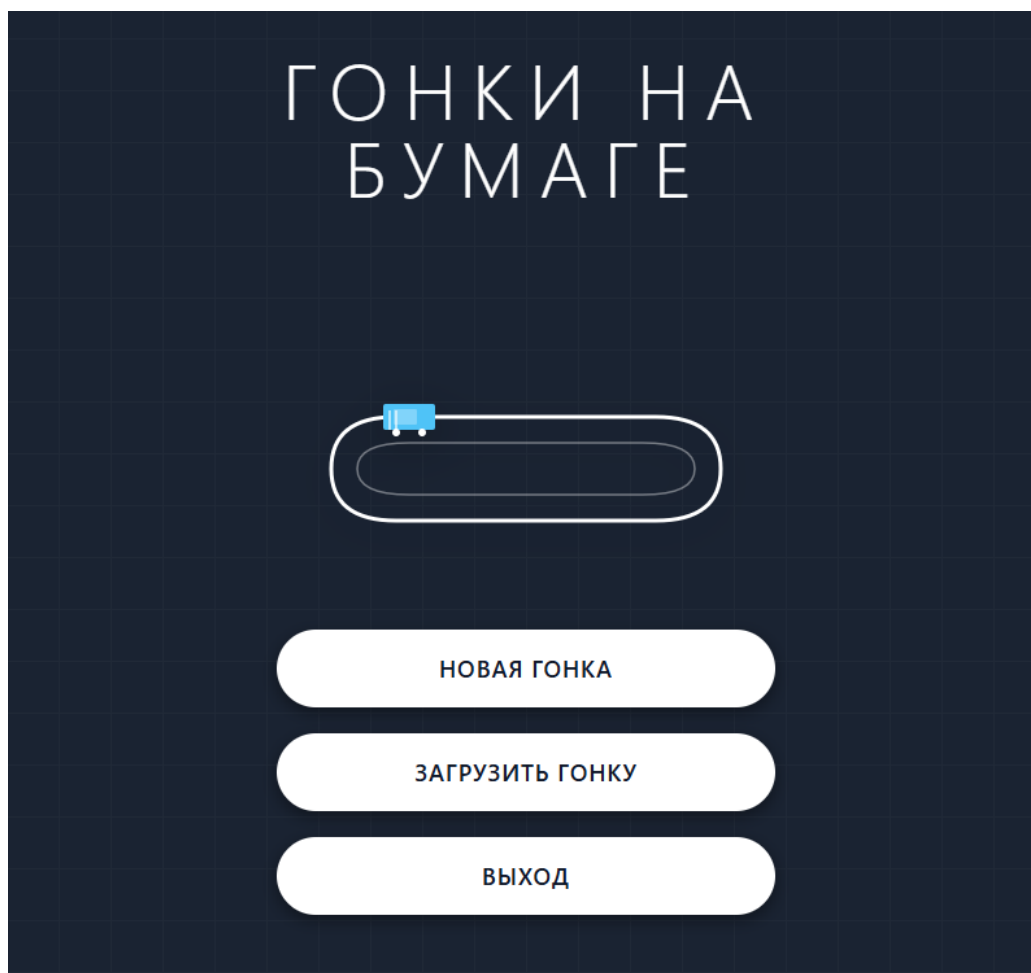


Рисунок 4.3.1 – Прототип интерфейса главного меню игры «Гонки на бумаге»

В данном разделе рассмотрены основные алгоритмы работы игры «Гонки на бумаге», включая навигацию по меню, выбор режимов игры и логику игрового процесса при игре с ботом или с другими пользователями, а также механизмы определения завершения игры и победителя.

Также представлен прототип пользовательского интерфейса, включающий главное меню, экраны настроек и игры, а также экран вывода результатов. Разработанные схемы алгоритма и прототип наглядно отражают логику работы системы и взаимодействие пользователя с приложением.

4.3 Разработка алгоритмов работы программы и программного обеспечения

В данном разделе представлена актуальная реализация программного обеспечения игры «Гонки на бумаге», рассмотрена структура программных модулей, описаны основные алгоритмы и пользовательский интерфейс приложения. В процессе доработки программы были уточнены экранные формы, добавлена полноценная работа с сохранениями, игровая логика вынесена в отдельный модуль правил, а алгоритм бота разделен на обычный и продвинутый уровни сложности.

Разработанное приложение реализует запуск игры, создание новой игровой сессии, выбор режима, выбор трассы, настройку участников, пошаговое перемещение автомобилей по клетчатому полю, автоматическую проверку столкновений, прохождение контрольных секторов, определение победителя, автосохранение состояния игры и загрузку ранее сохраненных партий.

4.3.1 Разработка программного обеспечения

Программное обеспечение игры «Гонки на бумаге» реализовано на языке Python с использованием библиотеки tkinter для построения графического пользовательского интерфейса. Для хранения пользовательских настроек, конфигураций трасс и игровых сохранений применяется формат JSON. Для загрузки изображений предварительного просмотра трасс используется библиотека Pillow. Архитектура программы построена по модульному принципу: отдельные файлы отвечают за запуск приложения, главное меню, настройки, создание новой игры, загрузку сохранений, игровой процесс, правила движения, управление ботом, конфигурацию интерфейса и хранение данных.

В текущей версии проекта используются следующие основные файлы: run.py, menu.py, settings.py, load_menu.py, new_game_menu.py, game.py,

core.py, save_manager.py, app_config.py, track_config.py, а также модули бота bot.py, bot_normal.py и bot_hard.py. Такое разделение позволяет независимо изменять пользовательский интерфейс, алгоритмы движения, систему сохранений и поведение программного соперника.

Для наглядности на рисунках 4.3.1-4.3.12 приведены структуры основных классов и модулей. В листингах отражены только ключевые методы и функции, определяющие назначение соответствующей части программы; вспомогательные методы интерфейса и отрисовки представлены обобщенно.

Главным классом приложения является класс Menu, расположенный в файле menu.py. Данный класс создает главное окно программы, переводит его в полноэкранный режим, загружает настройки оформления и языка, формирует главное меню и обеспечивает переходы между экранными формами приложения. Структура класса представлена на рисунке 4.3.1.

```
import tkinter as tk
from settings import Settings
from load_menu import LoadMenu
from new_game_menu import NewGameMenu
from app_config import load_config, save_config, get_theme, tr

class Menu:
    def __init__(self):
        ...
    def create_menu(self):
        ...
    def create_menu_buttons(self):
        ...
    def create_settings_button(self):
        ...
    def open_settings(self):
        ...
    def new_game(self):
        ...
    def load_game(self):
        ...
    def confirm_exit(self):
        ...
```

Рисунок 4.3.1 – Класс Menu

Метод `__init__(self)` создает корневое окно `tkinter`, задает заголовок, включает полноэкранный режим, определяет размеры экрана и загружает параметры из конфигурационного файла. Метод `create_menu(self)` формирует

стартовый экран с декоративной сеткой и траекторией. Метод `create_menu_buttons(self)` создает кнопки «Новая гонка», «Загрузить гонку» и «Выход». Метод `create_settings_button(self)` отображает кнопку-иконку перехода к настройкам. Методы `open_settings(self)`, `new_game(self)` и `load_game(self)` открывают соответствующие экранные формы, а метод `confirm_exit(self)` создает экран подтверждения выхода.

Класс `Settings`, расположенный в файле `settings.py`, отвечает за экран настроек. Он позволяет изменить язык интерфейса и цветовую тему оформления, визуально выделяет выбранные параметры и сохраняет их в конфигурационный файл. Структура класса представлена на рисунке 4.3.2.

```
import tkinter as tk

class Settings:
    def __init__(self, menu):
        ...
    def create_ui(self):
        ...
    def apply_selected_styles(self):
        ...
    def set_language(self, lang):
        ...
    def set_theme(self, theme):
        ...
    def back(self):
        ...
```

Рисунок 4.3.2 – Класс `Settings`

Метод `create_ui(self)` создает заголовок, разделы выбора языка и темы, кнопки «Русский», «English», «Синий», «Красный» и «Фиолетовый». Методы `set_language(self, lang)` и `set_theme(self, theme)` изменяют параметры объекта `Menu`, вызывают сохранение настроек и заново строят экран, чтобы изменения сразу отображались в интерфейсе. Метод `back(self)` возвращает пользователя в главное меню.

Класс `LoadMenu`, расположенный в файле `load_menu.py`, реализует экран загрузки игры. В отличие от ранней версии, загрузка сохранений реализована полностью: класс получает список сохраненных слотов через модуль `save_manager.py`, отображает доступные партии, позволяет загрузить

выбранную игру и удалить ненужное сохранение. Структура класса представлена на рисунке 4.3.3.

```
import tkinter as tk
from save_manager import list_saves, load_save, delete_save
from game import Game

class LoadMenu:
    def __init__(self, menu):
        ...
    def create_ui(self):
        ...
    def refresh_list(self):
        ...
    def load_game(self, slot):
        ...
    def delete_save(self, slot):
        ...
    def create_back_button(self):
        ...
```

Рисунок 4.3.3 – Класс LoadMenu

Метод `refresh_list(self)` обращается к функции `list_saves()` и формирует список из трех слотов сохранений. Для заполненных слотов отображаются режим игры и время сохранения, а для пустых слотов кнопки загрузки и удаления блокируются. Метод `load_game(self, slot)` считывает данные из JSON-файла с помощью `load_save(slot)` и создает игровой объект через `Game.from_save(self.menu, save_data)`. Метод `delete_save(self, slot)` удаляет файл сохранения и обновляет список на экране.

Класс `NewGameMenu`, расположенный в файле `new_game_menu.py`, реализует пошаговый сценарий создания новой игровой сессии. Пользователь выбирает режим игры, при выборе режима «Бот» указывает сложность, затем выбирает трассу и настраивает цвета участников. Структура класса представлена на рисунке 4.3.4.

```
import os
import tkinter as tk
from PIL import Image, ImageTk
from game import Game, load_track

class NewGameMenu:
    def __init__(self, menu):
        ...
    def show_mode_screen(self):
```

```

...
def show_track_screen(self):
...
def show_players_screen(self):
...
def select_mode(self, mode):
...
def select_difficulty(self, level):
...
def select_track(self, index):
...
def add_player(self):
...
def remove_player(self, index):
...
def select_color(self, player_index, color):
...
def start_game(self):
...
def next_step(self):
...
def go_back(self):
...

```

Рисунок 4.3.4 – Класс NewGameMenu

Метод `show_mode_screen(self)` отображает карточки «Локальный мультиплеер» и «Бот». При выборе режима с ботом на этом же экране появляется блок сложности с вариантами «Обычный» и «Продвинутый». Метод `show_track_screen(self)` строит экран выбора одной из четырех трасс и отображает изображения предварительного просмотра. Метод `show_players_screen(self)` формирует карточки участников: в режиме с ботом создаются «Игрок 1» и «Бот», а в локальном режиме доступно добавление игроков до четырех участников. Метод `start_game(self)` проверяет наличие выбранных цветов и их уникальность, загружает JSON-файл трассы и создает объект класса `Game`.

Класс `Car`, расположенный в файле `game.py`, представляет автомобиль игрока или бота. Объект хранит координаты, текущий вектор скорости, цвет, историю пройденных точек, состояние прохождения контрольных секторов и сведения о последнем столкновении. Структура класса представлена на рисунке 4.3.5.

```

class Car:
    def __init__(self, x, y, color):
        self.x = x

```

```

self.y = y
self.vx = 0
self.vy = 0
self.color = color
self.path = [(x, y)]
self.started = False
self.checkpoint_index = 0
self.just_crashed = False
self.crash_axis = None

```

Рисунок 4.3.5 – Класс Car

Поля *x* и *y* задают текущую точку автомобиля на клетчатом поле. Поля *vx* и *vy* задают текущий вектор скорости. Список *path* используется для отрисовки траектории движения. Поля *checkpoint_index* и *started* применяются при проверке корректного прохождения трассы и финиша. Поля *just_crashed* и *crash_axis* используются для обработки ограничений после столкновения со стеной.

Класс *Track*, также расположенный в файле *game.py*, описывает игровую трассу. Он преобразует данные, загруженные из JSON-файла, в структуры, удобные для быстрого поиска стен, финиша, контрольных секторов и стартовых позиций. Структура класса представлена на рисунке 4.3.6.

```

class Track:
    def __init__(self, data):
        self.width = data["width"]
        self.height = data["height"]
        self.start = [tuple(point) for point in data["start"]]
        self.finish = {tuple(point) for point in data["finish"]}
        self.walls = {tuple(point) for point in data["walls"]}
        self.spawn_positions = [tuple(point) for point in data["spawn_positions"]]
        self.checkpoints = [
            tuple(point) for point in checkpoint
            for checkpoint in data.get("checkpoints", [])
        ]

    def is_inside_bounds(self, x, y):
        ...
    def is_wall(self, x, y):
        ...
    def is_finish(self, x, y):
        ...

```

Рисунок 4.3.6 – Класс Track

Метод *is_inside_bounds(self, x, y)* проверяет, находится ли точка в пределах карты. Метод *is_wall(self, x, y)* определяет, является ли клетка стеной. Метод *is_finish(self, x, y)* проверяет принадлежность клетки финишной

зоне. Функция `load_track(path)` загружает файл трассы и преобразует координаты в кортежи.

Основным классом игрового процесса является класс `Game`, расположенный в файле `game.py`. Он создает игровое поле, размещает автомобили, отвечает за отрисовку трассы, обработку кликов пользователя, выполнение ходов, работу бота, автосохранение и отображение результата игры. Структура класса представлена на рисунке 4.3.7.

```
from bots.bot import choose_bot_move
from save_manager import write_save_to_slot, build_save_meta, push_new_save_to_top,
promote_save_to_top, delete_save
from core import check_collision_path, check_collision_path_info, check_checkpoint,
check_finish, get_possible_moves, apply_move

class Game:
    def __init__(self, menu, players, track_data, mode="local", difficulty=None,
                 track_path=None, loaded_state=None, save_slot=None):
        ...
    def create_ui(self):
        ...
    def serialize_state(self):
        ...
    def restore_state(self, state):
        ...
    def auto_save(self):
        ...
    @classmethod
    def from_save(cls, menu, save_data):
        ...
    def make_bot_move(self):
        ...
    def draw(self):
        ...
    def on_click(self, event):
        ...
    def perform_move(self, gx, gy):
        ...
    def next_player(self):
        ...
    def show_win_screen(self, player_index):
        ...
```

Рисунок 4.3.7 – Класс `Game`

Метод `__init__(self, ...)` получает параметры выбранной игры, создает объект `Track`, рассчитывает размеры игрового поля, выбирает стартовые позиции и создает автомобили участников. Если игра открывается из сохранения, вызывается метод `restore_state(self, state)`, восстанавливающий координаты, скорости, траектории, активного игрока и состояние

контрольных точек. Метод `create_ui(self)` создает холст `Canvas`, привязывает обработчик щелчка мыши и запускает отрисовку. В текущей версии выбор хода выполняется не отдельной кнопкой, а щелчком по одному из подсвеченных маркеров допустимого перемещения.

Метод `draw(self)` очищает холст и последовательно отрисовывает сетку, стены, финиш, контрольные сектора, траектории, автомобили, возможные ходы и кнопку-иконку возврата в главное меню. Метод `on_click(self, event)` переводит координаты щелчка мыши в координаты сетки и выполняет ход только в том случае, если выбранная точка входит в список допустимых ходов. Метод `perform_move(self, gx, gy)` вызывает `apply_move()` из модуля `core.py`, сохраняет состояние игры, проверяет победу и передает очередь следующему участнику.

Методы `serialize_state(self)`, `restore_state(self, state)`, `auto_save(self)` и `from_save(cls, menu, save_data)` обеспечивают сохранение и загрузку игры. В сохраняемое состояние входят режим игры, сложность, данные трассы, номер текущего игрока, список участников, координаты и скорости автомобилей, их траектории, прогресс прохождения контрольных секторов и сведения о столкновениях.

Модуль `core.py` содержит основную математическую и игровую логику. В него вынесены функции проверки пересечения отрезков со стенами, определения точки останова при столкновении, вычисления допустимых ходов, проверки контрольных секторов и финиша, а также применения хода к автомобилю. Структура модуля представлена на рисунке 4.3.8.

```
from dataclasses import dataclass
from typing import Optional

@dataclass
class MoveResult:
    collision: bool
    final_x: int
    final_y: int
    new_vx: int
    new_vy: int
    hit_axis: Optional[str] = None

@dataclass
```

```

class TurnResult:
    collision: bool
    checkpoint_now: bool
    finish_now: bool
    final_x: int
    final_y: int
    new_vx: int
    new_vy: int
    hit_axis: Optional[str] = None

def check_collision_path_info(walls, x1, y1, x2, y2):
    ...
def get_possible_moves(track, car, max_speed):
    ...
def check_checkpoint(track, car, nx, ny):
    ...
def check_finish(track, car, nx, ny, checkpoint_now=False):
    ...
def apply_move(track, car, gx, gy):
    ...

```

Рисунок 4.3.8 – Модуль core.py

Функция `get_possible_moves(track, car, max_speed)` перебирает варианты ускорения по осям X и Y в диапазоне от -1 до 1. Для каждого варианта рассчитываются новые компоненты скорости и конечные координаты. Ходы, выходящие за границы поля или превышающие максимальную скорость, отбрасываются. В программе дополнительно используется ограничение `max_speed`, равное 5, что предотвращает слишком большие перемещения за один ход.

Функция `check_collision_path_info(walls, x1, y1, x2, y2)` проверяет, пересекает ли отрезок движения клетку-стену. Если столкновение найдено, определяется момент первого контакта, безопасная точка остановки и ось столкновения. Функция `apply_move(track, car, gx, gy)` применяет ход: при столкновении автомобиль останавливается в безопасной точке, скорость сбрасывается до нуля, а при обычном ходе обновляются координаты, скорость и траектория. После обычного хода дополнительно проверяется прохождение контрольного сектора и финиша.

Модуль `save_manager.py` реализует работу с файлами сохранений. Все сохранения хранятся в каталоге `saves` в формате JSON. Количество одновременно хранимых сохранений ограничено тремя слотами, что

соответствует требованиям технического задания. Структура модуля представлена на рисунке 4.3.9.

```
import json
import os
from datetime import datetime

SAVES_DIR = "saves"
MAX_SAVES = 3

def list_saves():
    ...
def load_save(slot):
    ...
def delete_save(slot):
    ...
def write_save_to_slot(slot, data):
    ...
def push_new_save_to_top(data):
    ...
def promote_save_to_top(slot, data):
    ...
def build_save_meta(name=None, track_name=""):
    ...
```

Рисунок 4.3.9 – Модуль save_manager.py

Функция `list_saves()` формирует список из трех слотов и возвращает сведения о каждом сохранении: номер слота, путь к файлу, название, дату и время, название трассы и режим игры. Функция `load_save(slot)` считывает JSON-файл выбранного слота. Функция `delete_save(slot)` удаляет файл сохранения. Функции `push_new_save_to_top(data)` и `promote_save_to_top(slot, data)` обеспечивают размещение последнего сохранения в первом слоте и сдвиг более старых сохранений вниз по списку.

Модуль `app_config.py` реализует хранение и обработку настроек интерфейса. В нем определены стандартные параметры запуска, цветовые темы, словари локализованных текстов и функции работы с конфигурационным файлом `config.json`. Структура модуля представлена на рисунке 4.3.10.

```
CONFIG_FILE = "config.json"

DEFAULT_CONFIG = {
    "theme": "blue",
    "language": "ru"
```

```

}

THEMES = {
    "blue": {...},
    "red": {...},
    "purple": {...}
}

TEXTS = {
    "ru": {...},
    "en": {...}
}

def load_config():
    ...
def save_config(config):
    ...
def get_theme(theme_name):
    ...
def tr(language, key, **kwargs):
    ...

```

Рисунок 4.3.10 – Модуль app_config.py

Словарь DEFAULT_CONFIG содержит значения по умолчанию: синюю тему и русский язык. Словарь THEMES описывает цвета фона, панелей, текста, кнопок и акцентов для синей, красной и фиолетовой тем. Словарь TEXTS хранит русские и английские подписи интерфейса. Функция load_config() загружает сохраненные настройки и проверяет их корректность, функция save_config(config) записывает настройки в файл, функция get_theme(theme_name) возвращает выбранную цветовую схему, а функция tr(language, key, **kwargs) возвращает локализованный текст.

Алгоритм программного соперника разделен на несколько модулей. Файл bot.py является маршрутизатором и содержит общие вспомогательные функции оценки расстояния до цели, опасности стен и направления движения. Файл bot_normal.py реализует обычный уровень сложности, а файл bot_hard.py – продвинутый уровень, использующий моделирование ходов вперед. Структура модулей представлена на рисунке 4.3.11.

```

# bot.py
def choose_bot_move(game, car, difficulty="normal"):
    if difficulty == "hard":
        from .bot_hard import choose_hard_move
        return choose_hard_move(game, car)
    from .bot_normal import choose_normal_move
    return choose_normal_move(game, car)

```



```

def distance_to_target(game, car, x, y, checkpoint_now=False):
    ...
def wall_danger(game, x, y):
    ...
def direction_bonus(game, car, x2, y2, checkpoint_now=False):
    ...

# bot_normal.py
def choose_normal_move(game, car):
    ...
def evaluate_normal_move(...):
    ...

# bot_hard.py
def choose_hard_move(game, car):
    ...
def simulate_move(game, base_car, gx, gy):
    ...

```

Рисунок 4.3.11 – Модули управления ботом

Обычный бот перебирает доступные ходы, разделяет их на безопасные и рискованные, оценивает продвижение к текущей цели, близость к стенам, скорость, повторное посещение клеток, столкновения, прохождение контрольной точки и достижение финиша. Продвинутый бот использует имитацию будущих ходов: для первого хода строятся возможные продолжения, применяется ограниченный поиск в глубину, после чего выбирается первый ход из наиболее перспективной ветви. Такой подход делает поведение продвинутого бота более осмысленным на поворотах и перед контрольными секторами.

Модуль `track_config.py` содержит функции для работы с конфигурационным списком трасс, а файл `run.py` является точкой входа в приложение. Структура данных модулей представлена на рисунке 4.3.12.

```

# track_config.py
TRACKS_CONFIG_PATH = os.path.join("tracks", "tracks_config.json")

def build_preview_path(track_file_path):
    ...
def load_tracks_config():
    ...

# run.py
from menu import Menu

if __name__ == "__main__":
    Menu()

```

Рисунок 4.3.12 – Модули track_config.py и run.py

Функция `load_tracks_config()` загружает список трасс из файла `tracks_config.json` и формирует для каждой трассы путь к файлу предпросмотра. Файл `run.py` содержит минимальный код запуска: при прямом запуске программы создается объект `Menu`, после чего управление передается главному циклу `tkinter`.

Так же на рисунке 4.3.11 представлена диаграмма классов, отражающая взаимосвязи между основными объектами программного обеспечения: классом главного меню, экранными классами, игровыми классами и служебными модулями.

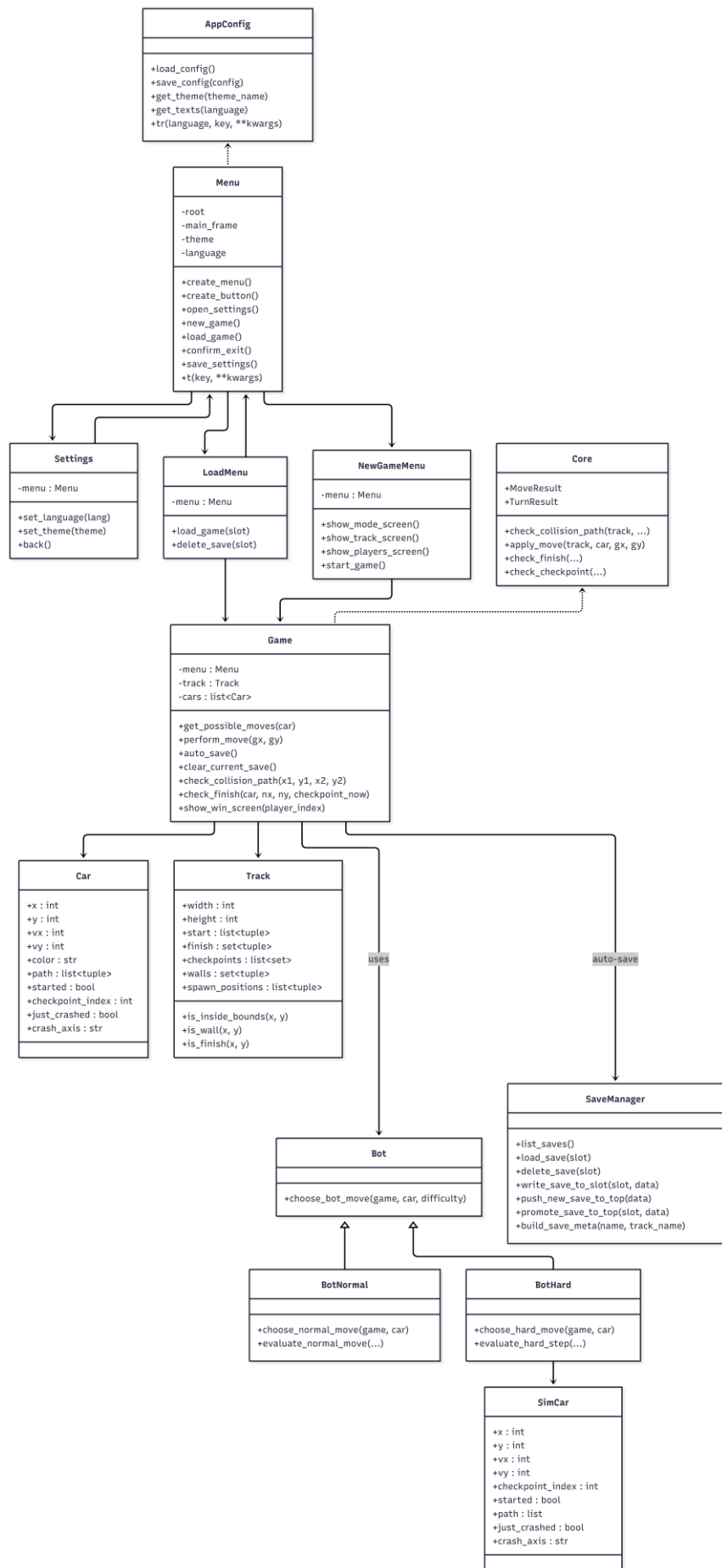


Рисунок 4.3.13 – Диаграмма классов

4.3.2 Разработка пользовательского интерфейса

Пользовательский интерфейс разработан с использованием библиотеки `tkinter`. Все экранные формы выполнены в едином стиле и зависят от выбранной цветовой темы. В программе реализованы три темы оформления: синяя, красная и фиолетовая, а также поддержка русского и английского языков. Выбранные параметры сохраняются в файле `config.json`.

Основными элементами интерфейса являются `Canvas`, `Frame`, `Label` и `Button`. `Canvas` используется для отображения фоновой сетки, декоративных элементов и игрового поля. `Frame` применяется для группировки элементов, `Label` - для заголовков и подписей, `Button` – для навигации и выбора параметров. Также в интерфейсе используются текстовые иконки: шестеренка настроек, стрелка назад и домик для возврата в главное меню.

Главное меню создается методом `create_menu()` класса `Menu`. На нем отображаются заголовок игры, декоративная траектория, кнопки «Новая гонка», «Загрузить гонку», «Выход» и иконка настроек. Внешний вид главного меню представлен на рисунке 4.3.14.

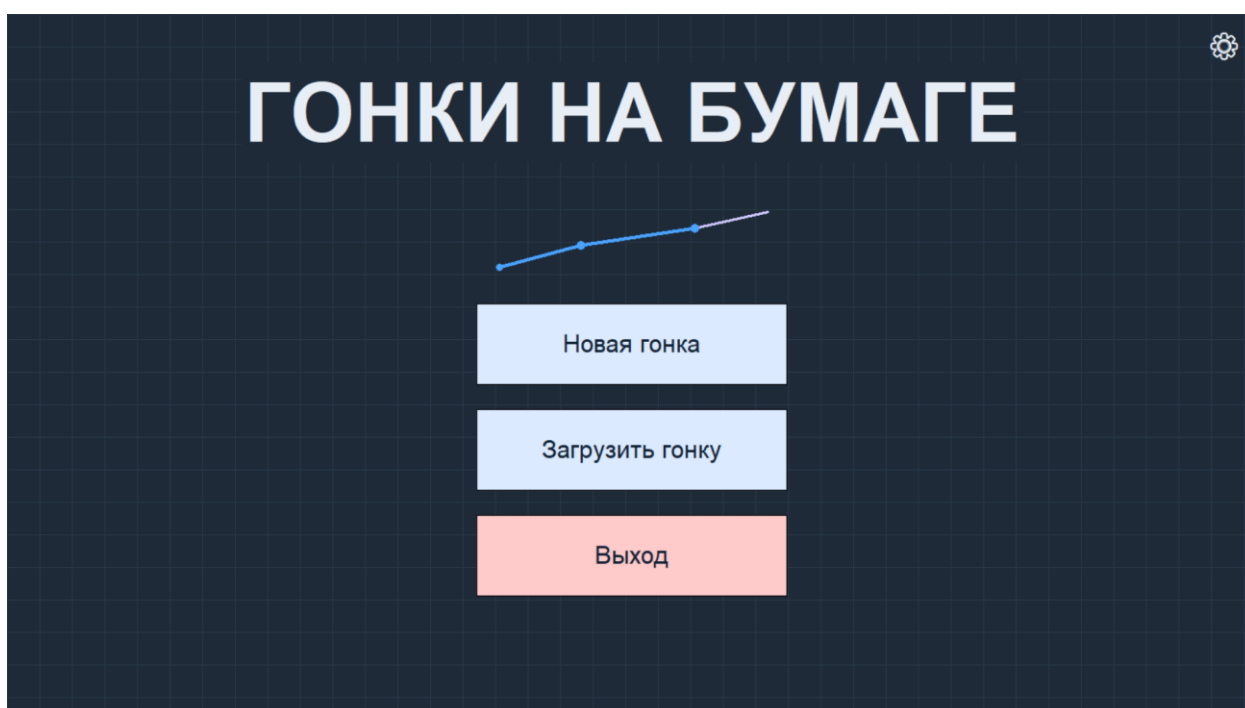


Рисунок 4.3.14 – Главное меню приложения

Экран подтверждения выхода формируется методом `confirm_exit()` класса `Menu`. На форме выводится вопрос о подтверждении завершения работы приложения и две кнопки: «Подтвердить» и «Отмена». При подтверждении главное окно закрывается, при отмене пользователь возвращается в главное меню. Экран подтверждения выхода представлен на рисунке 4.3.15.

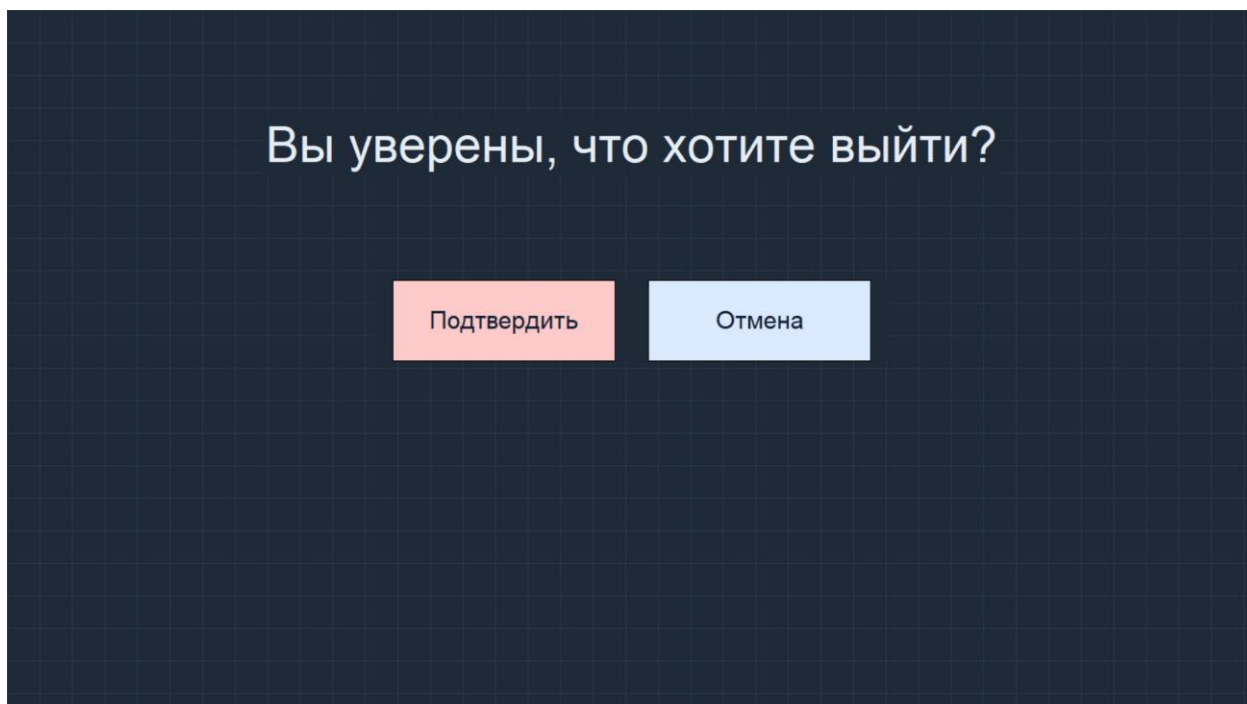


Рисунок 4.3.15 – Экран подтверждения выхода

Экран настроек создается классом Settings. Он содержит заголовок, блок выбора языка, блок выбора оформления и кнопку закрытия в правом верхнем углу. Пользователь может выбрать русский или английский язык, а также синюю, красную или фиолетовую цветовую тему. Активные параметры визуально выделяются цветом выбранной кнопки. После изменения языка или темы экран сразу перестраивается, а параметры сохраняются в config.json. Экран настроек изображен на рисунке 4.3.16.

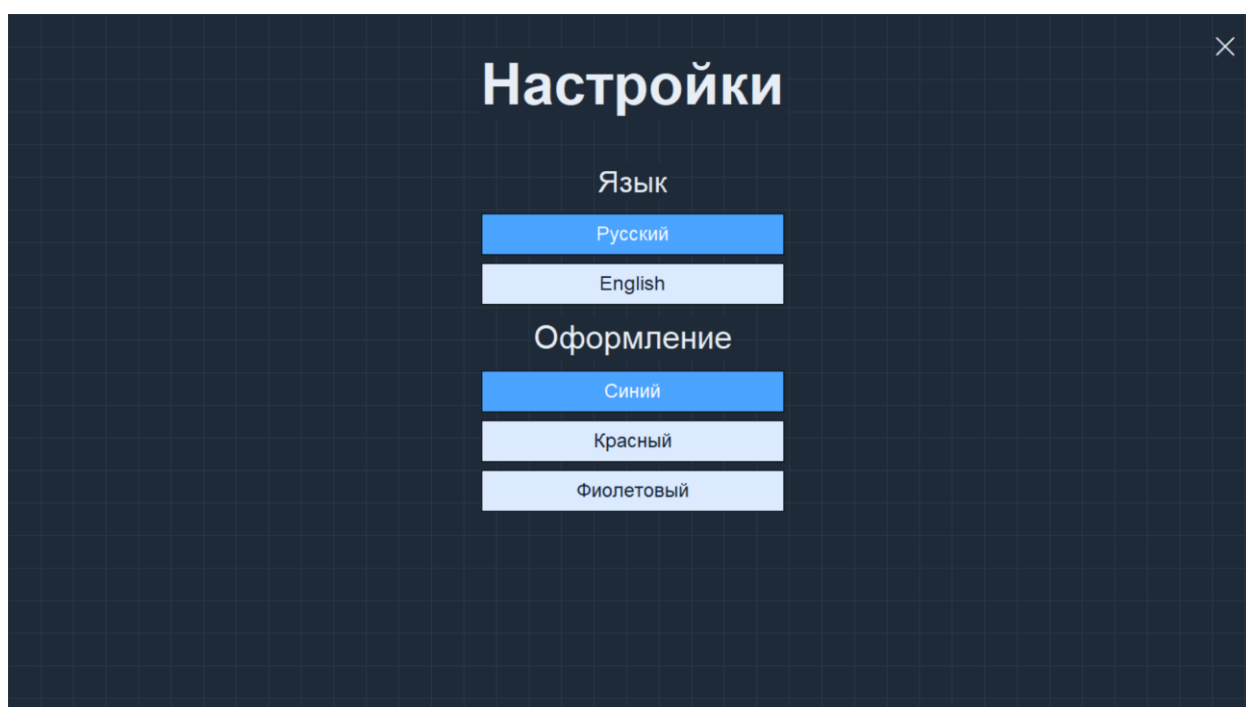


Рисунок 4.3.16 – Экран настроек

Экран загрузки игры реализован классом LoadMenu. На форме отображается заголовок и список трех слотов сохранений. Для каждого заполненного слота выводятся номер гонки, режим игры и время сохранения. Рядом расположены кнопки «Загрузить» и «Удалить». Если слот пустой, соответствующие кнопки становятся недоступными. При нажатии на «Загрузить» программа восстанавливает сохраненное состояние игры, а при нажатии на «Удалить» удаляет выбранный JSON-файл сохранения. Экран загрузки игры представлен на рисунке 4.3.17.

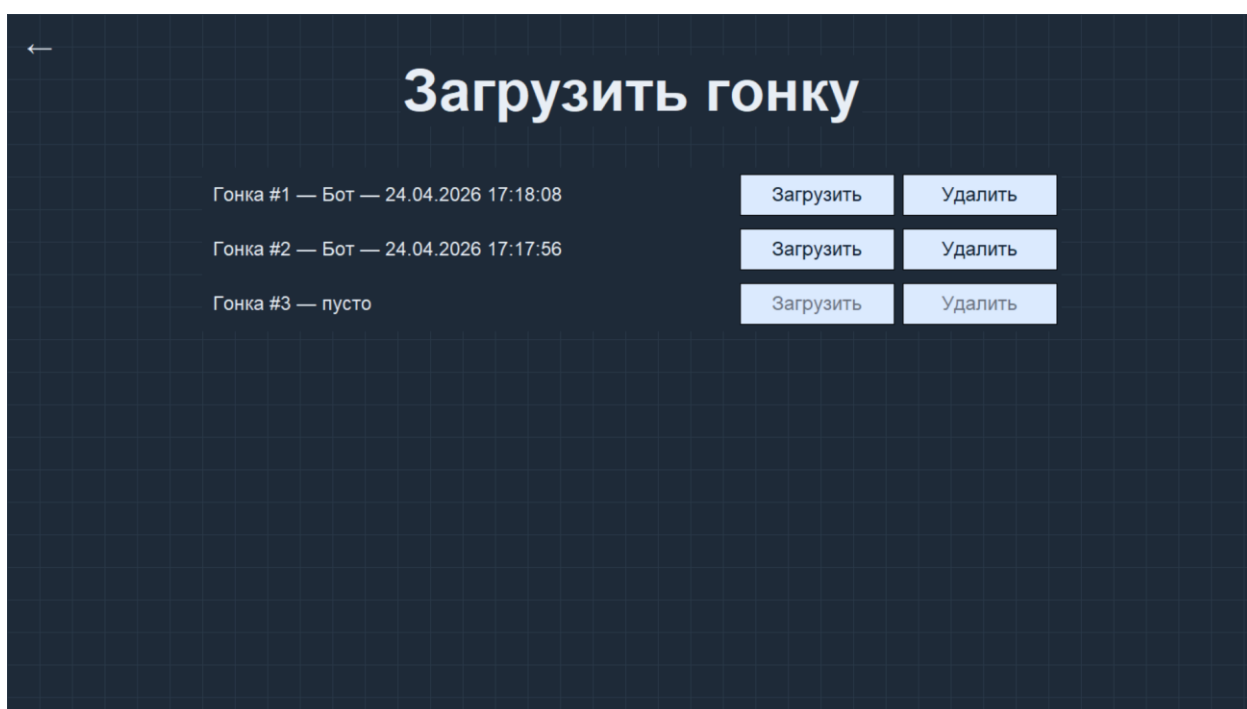


Рисунок 4.3.17 – Экран загрузки игры

Экран выбора режима игры реализован методом `show_mode_screen()` класса `NewGameMenu`. На экране отображаются две карточки: «Локальный мультиплеер» и «Бот». Выбранная карточка подсвечивается цветом активной темы. Если пользователь выбирает режим игры с ботом, ниже появляется блок выбора сложности с кнопками «Обычный» и «Продвинутый». Переход к следующему этапу выполняется кнопкой «Далее», а возврат в главное меню – иконкой стрелки назад. Экран выбора режима игры и сложности бота изображен на рисунке 4.3.18.

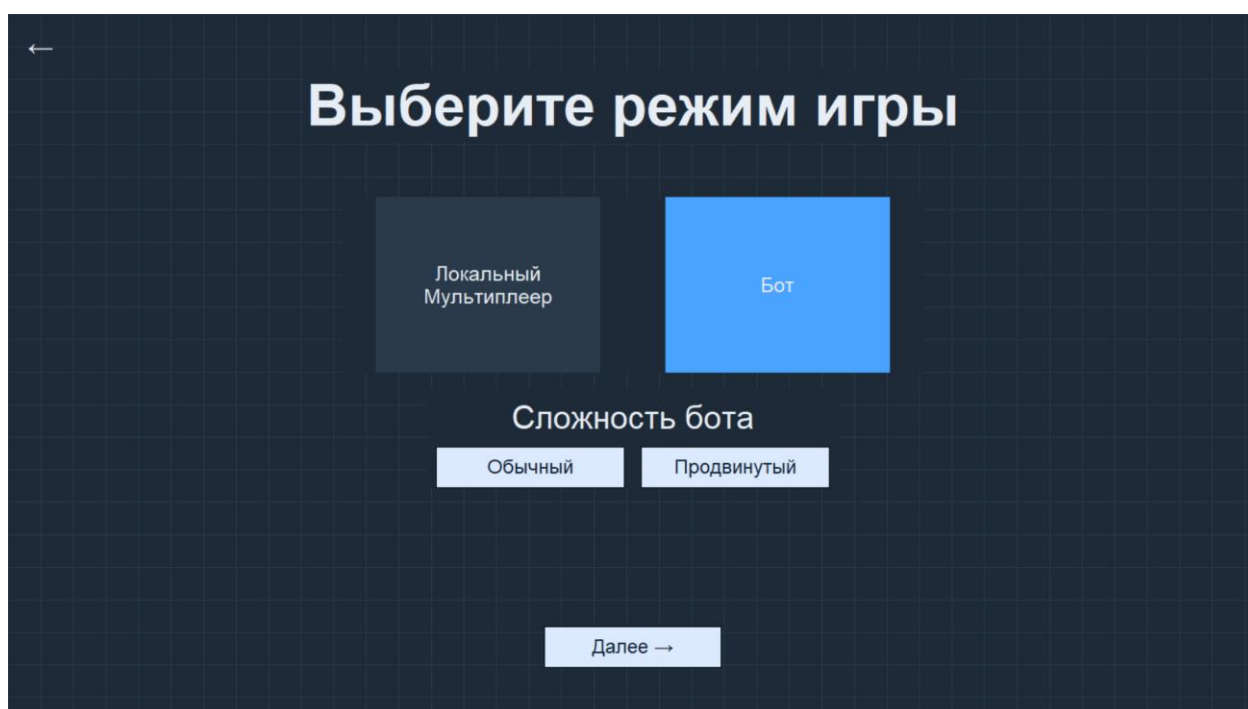


Рисунок 4.3.18 – Экран выбора режима игры и сложности бота

Экран выбора трассы создается методом `show_track_screen()` класса `NewGameMenu`. Он содержит четыре карточки трасс. Каждая карточка включает название трассы и область предварительного просмотра, изображение для которой загружается из файла PNG. Выбранная трасса выделяется изменением цвета карточки. Для перехода к настройке игроков используется кнопка «Далее». Экран выбора трассы представлен на рисунке 4.3.19.

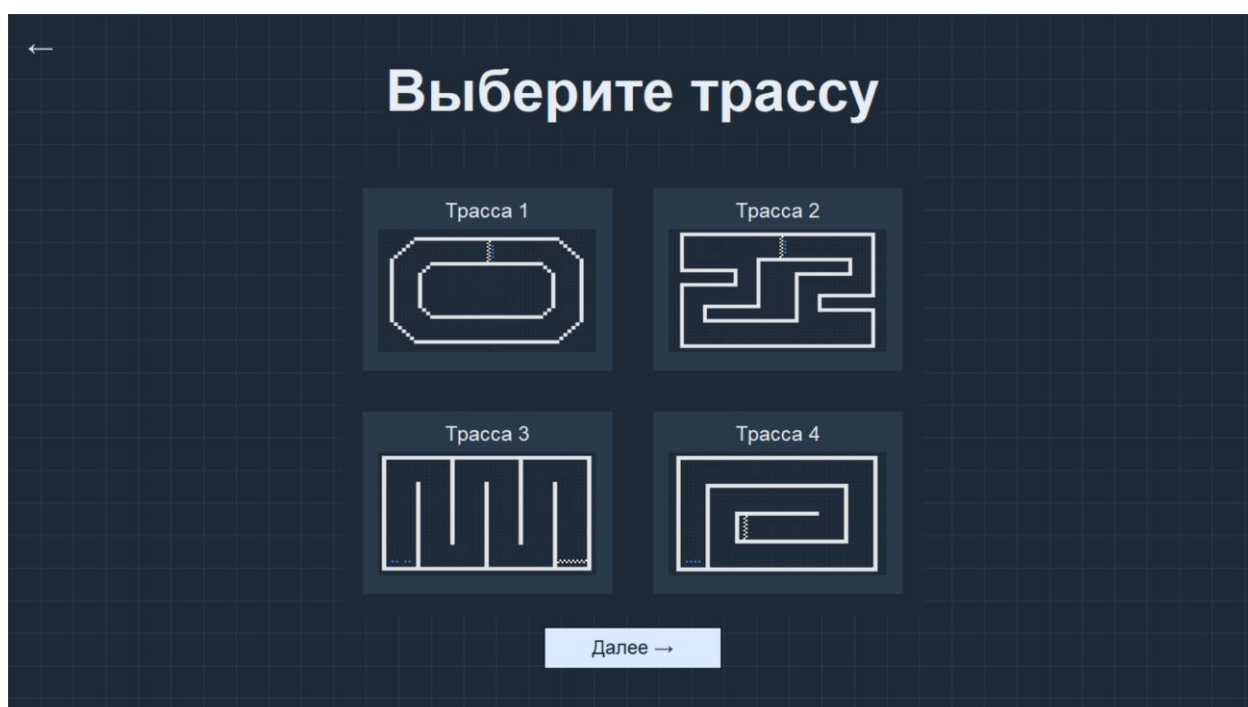


Рисунок 4.3.19 – Экран выбора трассы

Экран настройки игроков также реализован в классе NewGameMenu. В режиме «Бот» на форме отображаются две карточки: игрок и бот. В режиме локального мультиплеера отображаются карточки игроков, при этом пользователь может добавлять участников до четырех человек и удалять их, если игроков больше двух. Для каждого участника доступен выбор одного из четырех цветов автомобиля. Если цвет выбран не у всех участников или цвета совпадают, программа выводит сообщение об ошибке и не запускает игру. Запуск игровой сессии осуществляется кнопкой «Начать игру». Экран настройки игроков представлен на рисунке 4.3.20.

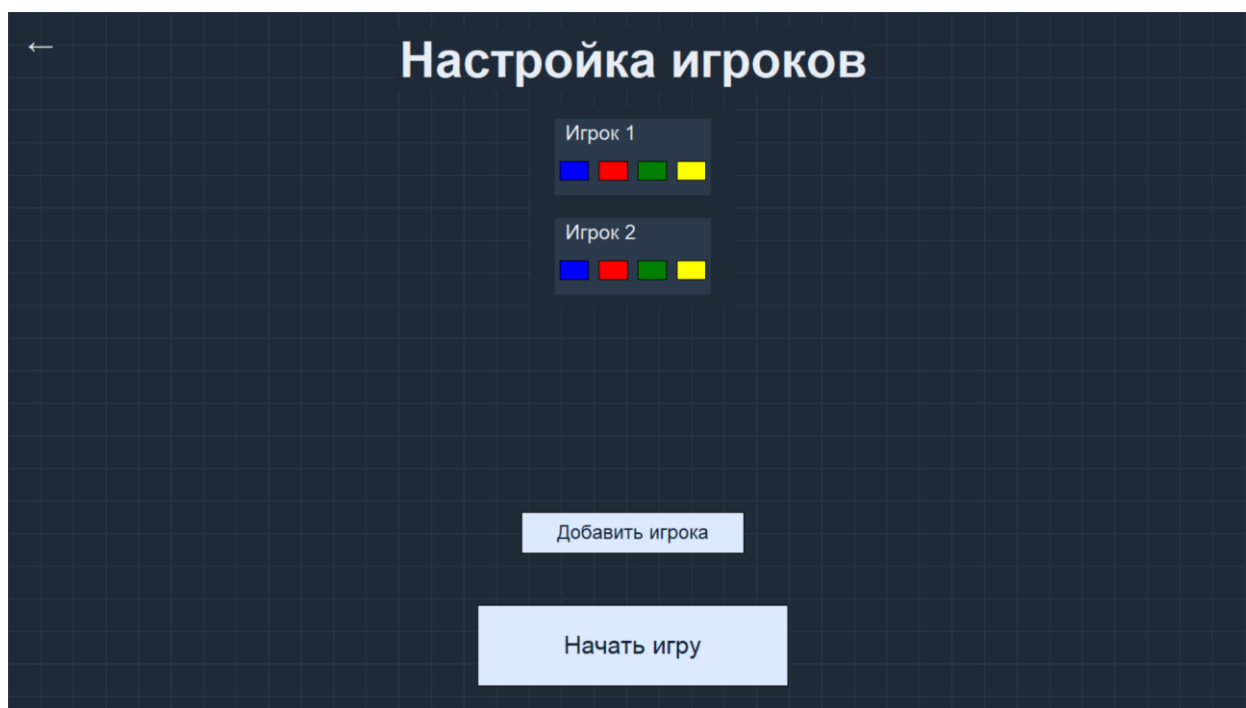


Рисунок 4.3.20 – Экран настройки игроков

Главный игровой экран создается классом Game. Основным элементом является холст Canvas, на котором отображается клетчатое поле, стены трассы, финишная зона, контрольные сектора, траектории автомобилей и текущие положения игроков. Допустимые варианты следующего хода отображаются небольшими маркерами цвета текущего игрока. Для выполнения хода пользователь нажимает на один из этих маркеров. В правом верхнем углу расположен значок домика, позволяющий выйти в главное меню с автосохранением текущей партии. Главный игровой экран изображен на рисунке 4.3.21.

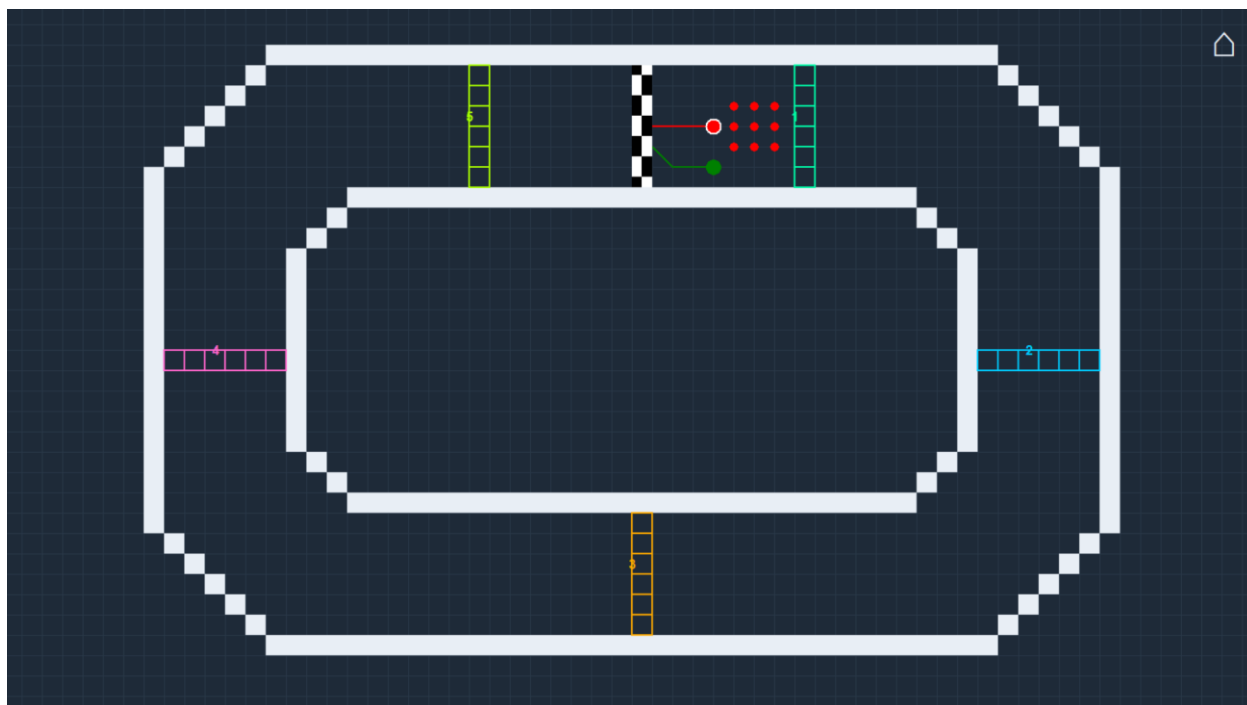


Рисунок 4.3.21 – Главный игровой экран

Экран результатов формируется методом `show_win_screen()` класса `Game`. На нем отображается надпись «Победил», имя победителя и цветовой маркер выигравшего автомобиля. Если победил бот, вместо номера игрока выводится подпись «Бот». Ниже расположены кнопки «Новая гонка» и «Главное меню». Кнопка «Новая гонка» перезапускает игру с теми же параметрами, а кнопка «Главное меню» возвращает пользователя на стартовый экран. Экран результатов игры представлен на рисунке 4.3.22.

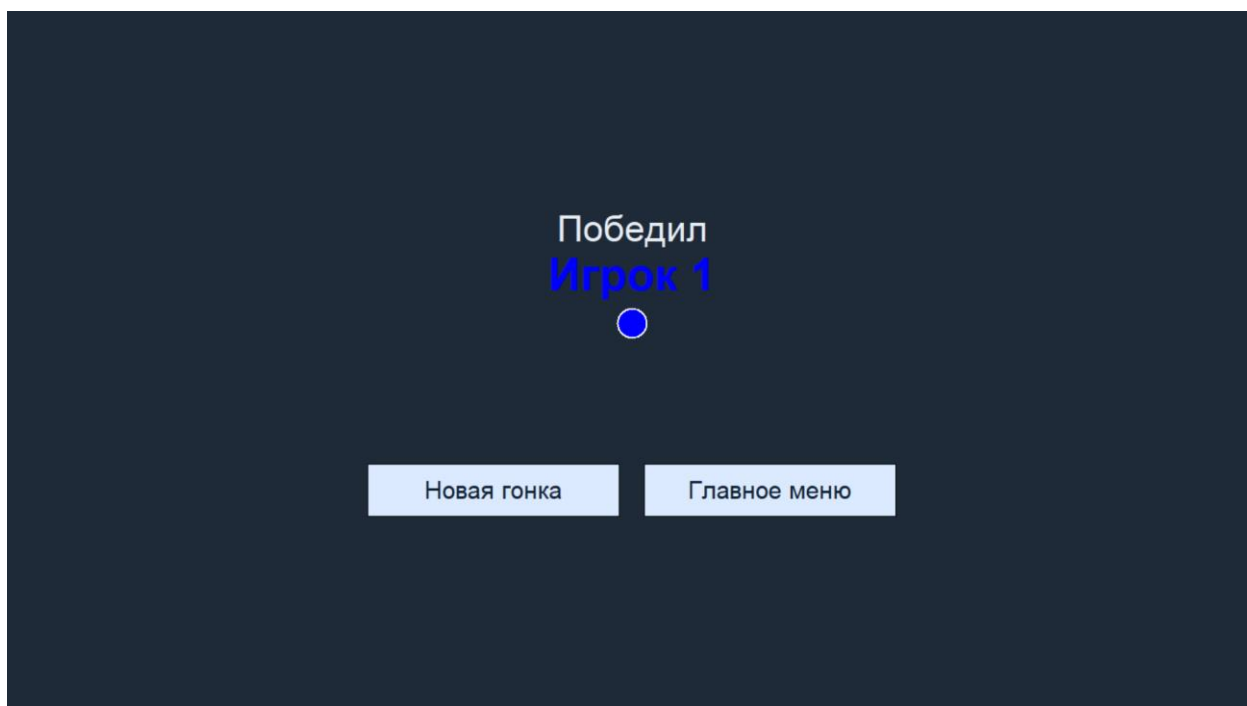


Рисунок 4.3.22 – Экран результатов игры

В результате выполнения данного раздела были разработаны алгоритмы функционирования игры «Гонки на бумаге», реализованы экранные формы пользовательского интерфейса, а также создана программная структура, включающая модули навигации, настройки параметров игры, игровой логики, работы бота и конфигурации интерфейса. Реализованная программа обеспечивает запуск приложения, создание новой игры, выбор режима, трассы и участников, отображение игрового процесса и определение победителя. Кроме того, реализована система хранения параметров интерфейса в конфигурационном файле JSON.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1) Преимущества и недостатки языка Python // URL: <https://www.hocktraining.com/blog/preimuschestva-yazyka-python> (дата обращения: 31.01.2026).
- 2) Что такое PyCharm // URL: <https://skyeng.ru/magazine/chtotakoe-pycharm/> (дата обращения: 31.01.2026).
- 3) Описание правил и механики игры «Гонки на бумаге» // URL: <https://smart-kids.su/igry/na-bumage/gonki> (Дата обращения: 24.01.2026)
- 4) Scientific article о Racetrack (алгоритмическое и математическое моделирование) // URL: <https://www.sciencedirect.com/science/article/pii/S0304397518302627> (Дата обращения: 24.01.2026)
- 5) Статья «Игра на бумаге – Гонки» (описание механики и примеры партий) // URL: <https://habr.com/ru/articles/18245/> (Дата обращения: 24.01.2026)
- 6) Пост «Настольная игра на бумаге – Гонки» (реальные примеры и обсуждение игры) // URL: https://pikabu.ru/story/nastolnaya_igra_na_bumage_gonki_10404072 (Дата обращения: 24.01.2026)
- 7) Необычные и интересные игры на бумаге (включая вариации гонок) // URL: <https://www.perunica.ru/raznoe/10093-neobychnye-i-interesnye-igry-na-bumage.html> (Дата обращения: 24.01.2026)